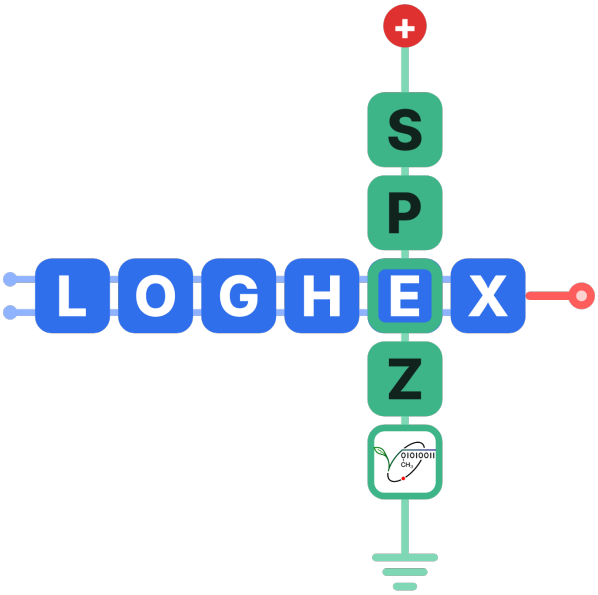


Handbuch

Simulation von Logikgattern, Halbleiterschaltungen
und einem Von-Neumann-Rechner für den Informatikunterricht

Johannes Süpke

MINT-Spezialklassen am Albert-Schweitzer-Gymnasium Erfurt

Erstellt für den Unterrichtseinsatz im Fach Informatik
mit Hilfe von Claude (KI-Assistent von Anthropic)

www.asgspez.de

Stand: November 2026
Lizenz: CC BY-SA 4.0 (ausgenommen die Logos), siehe Kap. 7

Inhaltsverzeichnis

0. Besonderheiten auf einen Blick **4**

1. Erste Schritte **8**
1.1. Programm starten . . . 8
1.2. Die Oberfläche . . . 9
1.3. Speichern, Öffnen und Teilen . . . 9
1.4. Einstellungen . . . 10
1.5. Tastatur . . . 11

2. Logikgatter **12**
2.1. Bauteile . . . 12
2.2. Simulation und Farben . . . 12
2.3. Flipflops . . . 12
2.4. Innerer Aufbau per Doppelklick . . . 13
2.5. Eigene Bausteine (modulares Arbeiten) . . . 13
2.6. Wahrheitstabelle, KV-Diagramm und Minimierung . . . 14
2.7. KV-Editor: Päckchen selbst bilden . . . 14
2.8. Schaltung aus einem Term erzeugen . . . 16
2.9. Impulsdiagramm (Messschreiber) . . . 17
2.10. Aufgabenmodus mit automatischer Prüfung . . . 18
2.11. Weitere Werkzeuge . . . 18

3. Elektronik & Halbleiter **19**
3.1. Bauteile . . . 19
3.2. Simulation und Darstellung . . . 19
3.3. Kennlinien und Oszilloskop . . . 20
3.4. Innerer Aufbau . . . 21

4. Halbleiter-Modell **22**
4.1. Wie sich Löcher bewegen . . . 22
4.2. Warum die Raumladungszone in Durchlassrichtung schmaler wird . . . 23

5. Rechner **24**
5.1. Anfänger- und Profi-Ansicht . . . 24
5.2. Aufbau . . . 25
5.3. Bitleitungen und Farben . . . 26
5.4. Steuerung . . . 27
5.5. Arbeitsspeicher und Hexcode . . . 27
5.6. Assembler . . . 28
5.7. Befehlssatz . . . 29
5.8. Pseudocode-Compiler (Hochsprache) . . . 29
5.9. Eigene Befehle per Torsteuerung . . . 35
5.10. MikroSim-Programme laden . . . 36
5.11. Erklärungen per Antippen . . . 36
5.12. Innerer Aufbau der Rechner-Bauteile . . . 37

6. Material für die Lehrkraft **39**
6.1. Beispieldateien laden und präsentieren . . . 39
6.2. Beispielschaltungen . . . 39
6.3. Beispielprogramme für den Rechner . . . 39

7. Über LOGHEX **41**

A. Qualitätssicherung **43**
A.1. Automatisierte Tests 43
A.2. Sicherheitsmaßnahmen 44
A.3. Prüfliste für das iPad 45

0. Besonderheiten auf einen Blick

LOGHEX ist eine eigenständige Lernumgebung. Sie vereint vier Bereiche in einem einzigen Programm: **Logikgatter**, **Elektronik & Halbleiter**, **Halbleiter-Modell** und **Rechner**. Die folgende Übersicht fasst alle Besonderheiten zusammen, die für den Unterricht entwickelt wurden. Die Verweise führen zu den ausführlichen Erklärungen.

Plattform und Einsatz

- **Läuft überall:** als Webanwendung im Browser (auch auf dem **iPad** mit Touch-Bedienung) und **offline unter Windows** als einzelne Datei `LOGHEX.html` – ohne Installation, ohne Anmeldung (Kap. 1).
- **Bedienung mit Finger oder Maus:** Doppelklick entspricht langem Tippen, Zoomen mit zwei Fingern oder dem Mausrad.
- **Beamer-Modus** mit dicken Linien und großer Schrift für die Präsentation im Klassenraum (Kap. 1.4).
- **Automatisches Speichern** im Browser, dazu Speichern und Öffnen von Dateien sowie Links zum Teilen.
- **Export** als Bild (PNG), Vektorgrafik (SVG) und Druck bzw. PDF – ideal für Arbeitsblätter und Tafelbilder.
- **Yenka-Dateien öffnen:** Vorhandene Schaltungen aus Yenka (`.yka`) – Logikschaltungen mit Gattern und eigenen Blöcken ebenso wie Elektronikschaltungen – werden mit Öffnen übernommen und lassen sich weiter simulieren und speichern.

Logikgatter (Kap. 2)

- Alle Grundgatter wahlweise in **DIN-** oder **ANSI-Schaltzeichen**; das NICHT-Gatter hat dieselben Maße wie UND und ODER.
- **RS-Flipflop ohne Takt und mit Takt** (taktzustandsgesteuert), außerdem D-, JK- und T-Flipflop.
- **Modulares Arbeiten:** Eigene Schaltungen werden zu **eigenen Bausteinen**, die man beliebig oft verwenden und ineinander verschachteln kann (Kap. 2.5).

- **Innerer Aufbau per Doppelklick** (oder langem Tippen): Ein Flipflop zeigt seinen Aufbau aus Logikgattern, ein Logikgatter seinen Aufbau aus **CMOS-Transistoren**. Man kann beliebig tief hineinzoomen und die Innenschaltung direkt ausprobieren (Kap. 2.4).
- **Wahrheitstabelle, KV-Diagramm und Minimierung** (Quine-McCluskey) auf Knopfdruck (Kap. 2.6).
- **KV-Editor:** KV-Diagramme (2 × 2, 2 × 4, 4 × 4) selbst ausfüllen, die **Päckchen selbst bilden** und **DNF bzw. KNF selbst aufschreiben**; LOGHEX prüft Päckchen und Terme und meldet, ob die Lösung minimal ist (Kap. 2.7).
- **Schaltungssynthese aus einem Term** – auch ausschließlich aus NAND- oder NOR-Gattern (Kap. 2.8).
- **Impulsdiagramm** (Messschreiber) zum Aufzeichnen von Signalen über der Zeit (Kap. 2.9).
- **Zeitlupe**, die die Gatterlaufzeit sichtbar macht – Laufzeiteffekte und Rückkopplungen werden erlebbar.
- **Prozessorbausteine:** Halb- und Volladdierer, 4-Bit-Addierer, 4-Bit-ALU, ALU-Scheibe, Register, Schieberegister, Zähler, RAM, Multiplexer, Decoder, Zahleneingabe, 8-Bit-Zahlenanzeige (Kap. 2.1).
- **Aufgabenmodus mit automatischer Prüfung:** Aus einer Musterlösung entsteht eine Aufgabendatei; die Schüler bauen und lassen ihre Lösung prüfen (Kap. 2.10).
- **Mehrfachauswahl, Kopieren und Einfügen**, Verschieben von Leitungspunkten, Rückgängig und Wiederholen.
- **Fehlerhinweise:** offene Eingänge, Kurzschlüsse zwischen Ausgängen und nicht zur Ruhe kommende Schaltungen werden gemeldet.

Elektronik und Halbleiter (Kap. 3)

- Echte Schaltungssimulation wie in SPICE (Knotenpotentialanalyse, Newton-Verfahren) mit Dioden, Z-Dioden, LEDs, Bipolar- und MOS-Transistoren.
- Weitere Bauteile: **Funktionsgenerator, Spule, Relais, Summer, Motor, Operationsverstärker, Timer 555, p-Kanal-MOSFET**, Fotowiderstand, Heißleiter u. a.
- Animierter **Stromfluss** (technische Stromrichtung oder Elektronenfluss) und farbige Potentiale.
- **Oszilloskop** und **Kennlinien** der Halbleiterbauteile.
- Auch Halbleiterbauteile zeigen per Doppelklick ihren inneren Aufbau.

Halbleiter-Modell (Kap. 4)

- Teilchenmodell von Eigenleitung, n- und p-Dotierung und **pn-Übergang** mit Raumladungszone, Durchlass- und Sperrrichtung sowie Kennlinie.

Rechner (Kap. 5)

- Simulation eines 8-Bit-Rechners nach dem **Von-Neumann-Prinzip** mit **Adress-, Daten- und Steuerbus**.
- **Anfänger- und Profi-Ansicht** des Rechners, sichtbar umschaltbar: vereinfacht mit einem Bus, Rechenwerk als Block und Erklärungen in Worten – oder vollständig mit allen Leitungen, Toren und Mikroschritten (Kap. 5.1).
- Programme werden als **Hexcode** direkt in den RAM geschrieben – oder in einer **Assemblersprache**, die im Hintergrund als Hexcode gespeichert wird.
- **Pseudocode-Compiler:** Programme in einer Hochsprache mit englischen Befehlen (IF, WHILE, FOR, REPEAT, PROCEDURE, ARRAY …) werden in Assembler übersetzt. Die Zuordnung Pseudocode → Assembler → Hexcode ist beim Ausführen live sichtbar; die Compiler-Phasen (Tokens, Syntaxbaum) und eine **Optimierung** mit Laufzeitvergleich lassen sich zeigen (Kap. 5.8).
- **Compiler-Labor:** Lexer (mit endlichem Automaten), Parser (Grammatik, Aufrufstapel, wachsender Syntaxbaum) und Codeerzeugung **Schritt für Schritt** beobachten – mit **Vorhersage-Modus**, in dem die Schüler den nächsten Schritt raten (Kap. 5.8).
- **Eigene Befehle per Torsteuerung** bauen (Mikroprogramme), **prüfen** lassen und gegen Änderungen **absichern** (Kap. 5.9).
- **MikroSim-Programme** (`.ras`, `.rad`) und Mikroprogramme (`.mpd`) laden und ausführen (Kap. 5.10).
- Alle Busse und Verbindungen (auch die beiden **ALU-Eingänge**) als **8 einzelne Leitungen**: Für jeden Befehl ist das **Bitmuster** sichtbar.
- Knopf Schritt: genau **ein Takt** (in der Anfänger-Ansicht: eine Phase, Befehl holen bzw. ausführen), danach wartet der Rechner auf die nächste Eingabe.
- Die ALU zeigt die gerade ausgeführte **Operation als Symbol** (+, −, ∧, ∨ …), ihre Ausgangsleitungen folgen der ALU-Form.
- **Dezimale Ein- und Ausgabe** (deutlich als dezimal gekennzeichnet), **scrollbarer RAM** in der Listenansicht mit hervorgehobener Zelle, auf die der **Adressbus** gerade zugreift.
- **Erklärungen per Antippen:** Jedes Bauteil zeigt, wofür seine Abkürzung steht und was es tut, dazu den aktuellen Inhalt (Kap. 5.11).

- **Innerer Aufbau per Doppelklick:** ALU, Register, Befehlszähler, Stapelzeiger, Flags, RAM, Bus und Steuerwerk springen in den Reiter **Logikgatter** und zeigen ihren Aufbau aus Gattern – mit den aktuellen Werten aus dem Rechner und sofort testbar (Kap. 5.12).

Für die Lehrkraft (Kap. 6)

- **Keine fertigen Beispiele im Programm** – die Schüler sollen selbst entwickeln. Die Beispielschaltungen und Beispielprogramme liegen als **separate Dateien** vor, die die Lehrkraft laden und präsentieren kann.
- **Geprüfte Qualität und Sicherheit:** umfangreiche automatische Tests, Eingabeprüfung aller Dateien und Links sowie eine Content-Security-Policy (Anhang A).
- Die **Hilfe** nennt die Entstehung des Programms und verlinkt auf www.asgspez.de. LOGHEX ist **freie Software** (EUPL 1.2), das Handbuch steht unter CC BY-SA 4.0 (Kap. 7).
- **17 Beispielprogramme in Assembler** (u. a. Lauflicht, Fibonacci, Division mit Rest, ein Programm, das sich selbst verändert) und **10 Beispielprogramme in Pseudocode** (u. a. ggT, Primzahlen, Bubblesort).

1. Erste Schritte

1.1. Programm starten

LOGHEX besteht aus einer einzigen Datei, `LOGHEX.html`. Sie enthält alles, was das Programm braucht, und lädt nichts aus dem Internet nach.

- **Windows (offline):** Datei auf den Rechner kopieren und doppelt anklicken. Sie öffnet sich im Standardbrowser (Edge, Chrome oder Firefox).
- **iPad und Tablets:** In Safari `loghex.asgspez.de` öffnen und über *Teilen* → *Zum Home-Bildschirm* als App ablegen. Nach dem ersten Aufruf läuft LOGHEX auch ohne Internet. (Die Datei `LOGHEX.html` selbst kann auf dem iPad nicht geöffnet werden: iPadOS zeigt lokale HTML-Dateien nur als Vorschau ohne JavaScript an.)
- **Schulserver:** Für einen eigenen Server gibt es ein fertiges Paket (`LOGHEX-Server.zip`) mit App-Symbol und Offline-Speicher; die beiliegende `ANLEITUNG.txt` beschreibt die Einrichtung. Über den Server funktionieren auch die Links zum Teilen.

1.2. Die Oberfläche

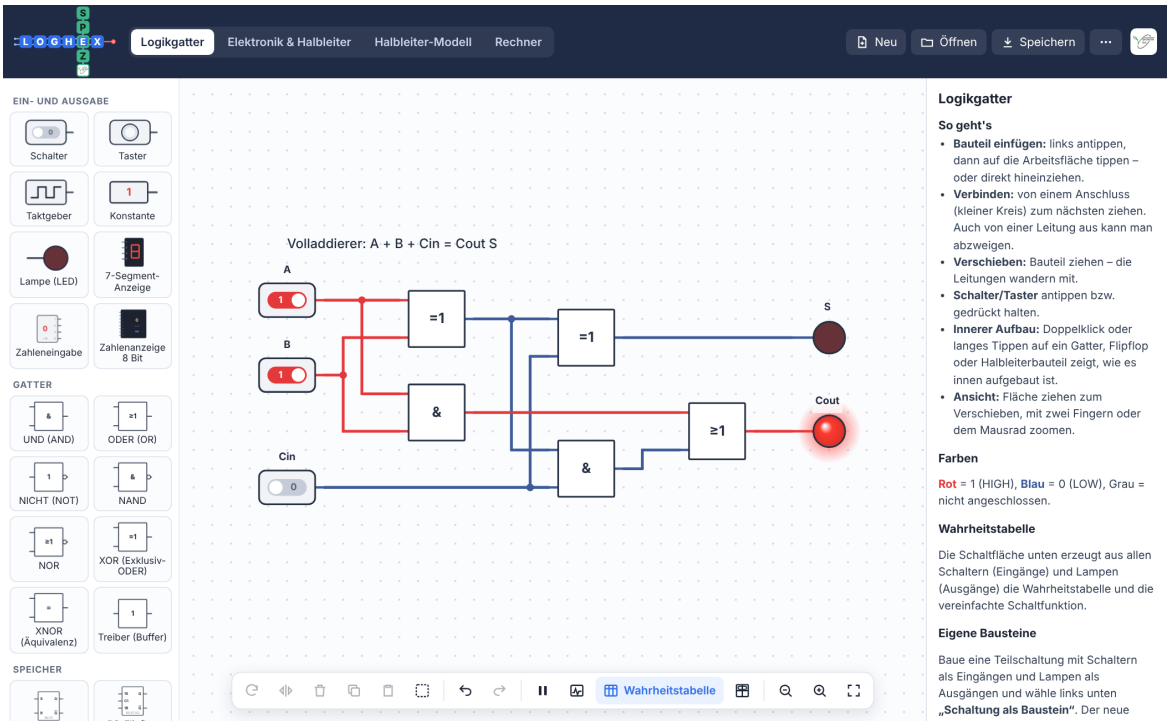


Abbildung 1.1.: Die Oberfläche im Reiter „Logikgatter“: Bauteilleiste links, Arbeitsfläche in der Mitte, Hilfe rechts, Werkzeugleiste unten.

Kopfzeile
Logo, die vier **Reiter** sowie Neu, Öffnen, Speichern und das Menü ··· (Export, Teilen, Werkzeuge, Einstellungen, **Handbuch** und Hilfe). Ganz rechts führt das Schullogo zur Website www.asgspez.de.

Bauteilleiste (links)
Bauteil antippen und dann auf die Arbeitsfläche tippen – oder direkt hineinziehen.

Arbeitsfläche
Bauteile verbinden, indem man von einem Anschluss (kleiner Kreis) zum nächsten zieht. Auch von einer Leitung aus kann man abzweigen. Fläche ziehen verschiebt die Ansicht; Mausrad oder zwei Finger zoomen.

Seitenleiste (rechts)
Hilfe, bzw. die Eigenschaften des gewählten Bauteils mit Messwerten.

Werkzeugleiste (unten)
Drehen, Spiegeln, Löschen, Kopieren, Einfügen, Auswahl, Rückgängig/Wiederholen, Pause, Messschreiber, Wahrheitstabelle, Zoom und „Alles zeigen“.

1.3. Speichern, Öffnen und Teilen

Alles wird **automatisch im Browser gespeichert**; beim nächsten Öffnen ist die Arbeit wieder da. Mit Speichern wird eine Datei (`.json`) heruntergeladen, die man mit Öffnen

wieder laden, per Mail verschicken oder in der Lernplattform ablegen kann. Über ⋯ → *Link zum Teilen* entsteht ein Link, der die ganze Schaltung enthält.

Yenka-Dateien übernehmen

Mit Öffnen lassen sich auch **Yenka-Dateien** (`.yka`) laden. LOGHEX liest Bauteile, Leitungen und Beschriftungen aus und baut die Schaltung im passenden Reiter nach: Logikschaltungen im Reiter *Logikgatter*, Elektronikschaltungen im Reiter *Elektronik & Halbleiter*.

- **Logik:** Gatter werden an ihrer IC-Nummer erkannt (z. B. 7408/4081 UND, 7432/4071 ODER, 7486 XOR, 7400 NAND, 7402 NOR, 7404/4069 NICHT – auch mit 3 oder 4 Eingängen wie 4073, 4072). *Logic Input* wird zum Schalter, *Logic Indicator* zur Lampe (Logikausgang), der Logik-Taster zum Taster, der Taktgeber zum Taktgeber und eine +5-V-Schiene zur Konstante 1. Der **4013** wird zum D-Flipflop, der **4008** zum 4-Bit-Addierer.
- **Yenka-Blöcke** (eigene Blöcke mit innerer Schaltung) werden zu **eigenen Bausteinen**; gleiche Blöcke ergeben einen gemeinsamen Baustein. Die Anschlüsse heißen E1, E2 … und A1, A2 …; Ein- und Ausgänge werden aus der inneren Schaltung bestimmt.
- **Elektronik:** Spannungsquellen und Spannungsschienen, Masse, Widerstände, Schalter und Taster, Dioden, LEDs, NPN- und PNP-Transistoren, Kondensatoren, Lampen.

Weil die Bauteile in LOGHEX kleiner sind, werden ihre Anschlüsse mit kurzen Leitungsstücken verlängert; die Verbindungen bleiben dabei vollständig erhalten. Unbekannte Bauteile werden mit „?“ markiert und im Hinweisfenster aufgelistet. Anschließend lässt sich die Schaltung mit Speichern als LOGHEX-Datei sichern.

Über das Menü ⋯ lassen sich Schaltungen außerdem **als PNG-Bild**, **als SVG-Vektorgrafik** und über *Drucken / als PDF speichern* ausgeben – praktisch für Arbeitsblätter.

1.4. Einstellungen

Unter ⋯ → *Einstellungen*:

- **Schaltzeichen:** DIN EN 60617 (rechteckig) oder ANSI/US (Formsymbole).
- **Zeitlupe für Gatter:** Jedes Gatter braucht sichtbar 0,3 s – Gatterlaufzeiten und Störimpulse werden beobachtbar.
- **Stromrichtung der Punkte:** technisch (+ → −) oder Elektronenfluss (− → +); Stromfluss animieren; Potentiale farbig.
- **Beamer-Modus:** dicke Linien und große Schrift.
- **Hinweise zur Fehlersuche**, **Ton für den Summer**, **Werte an Bauteilen**, **Simulationstempo** (Zeitlupe bis Zeitraffer).

1.5. Tastatur

R / M	Bauteil drehen / spiegeln
Entf	Löschen
Strg+Z / Strg+Y	Rückgängig / Wiederholen
Strg+C, Strg+V, Strg+D	Kopieren, Einfügen, Duplizieren
Strg+A	Alles auswählen
Umschalt+Ziehen	Mehrere Bauteile mit einem Rahmen auswählen
Esc	Abbrechen bzw. Infofenster schließen
Leertaste / →	(Rechner) ein Takt
Eingabe	(Rechner) ein ganzer Befehl

2. Logikgatter

2.1. Bauteile

Ein- und Ausgabe	Schalter, Taster, Taktgeber, Konstante, Lampe (LED), 7-Segment-Anzeige, Zahleneingabe (4 Bit), Zahlenanzeige 8 Bit (dezimal und hexadezimal)
Gatter	UND, ODER, NICHT, NAND, NOR, XOR, XNOR, Treiber – mit 2 bis 4 Eingängen
Speicher	RS-Flipflop, RS-Flipflop getaktet, D-, JK- und T-Flipflop, 4-Bit-Register, Schieberegister, 4-Bit-Zähler, RAM 16 × 4 Bit
Schaltnetze	Multiplexer 2:1 und 4:1, Decoder 2 zu 4
Rechenwerk	Halbaddierer, Volladdierer, 4-Bit-Addierer, 4-Bit-ALU, ALU-Scheibe (1 Bit)
Sonstiges	Beschriftung (Text auf der Arbeitsfläche)

Eigenschaften wie die Taktfrequenz oder die Farbe einer Lampe stellt man in der Seitenleiste ein, nachdem man das Bauteil angetippt hat. Bei einem ausgewählten Gatter erscheint direkt darunter der Knopf `+ Eingang`: Damit bekommt UND, ODER, NAND, NOR, XOR oder XNOR einen dritten bzw. vierten Eingang, mit `−` wird wieder einer entfernt. Bereits angeschlossene Leitungen bleiben dabei verbunden.

2.2. Simulation und Farben

Leitungen mit dem Wert **1 (HIGH)** sind rot, Leitungen mit **0 (LOW)** blau, nicht angeschlossene Leitungen grau. Offene Eingänge gelten als 0. Die Simulation läuft ständig mit; mit `Pause` lässt sie sich anhalten.

Die **Fehlerhinweise** melden u. a. nicht angeschlossene Eingänge, zwei direkt verbundene Ausgänge mit unterschiedlichen Werten (Kurzschluss) und Rückkopplungen, die nicht zur Ruhe kommen.

2.3. Flipflops

Das **RS-Flipflop** (Basis-Flipflop) speichert ein Bit: S setzt, R setzt zurück. Das **getaktete RS-Flipflop** übernimmt S und R nur, solange der Takteingang C = 1 ist (taktzustandsgesteuert). D-, JK- und T-Flipflop sind taktflankengesteuert. Der verbotene Zustand S = R = 1 wird angezeigt.

2.4. Innerer Aufbau per Doppelklick

Ein **Doppelklick** (oder **langes Tippen**) auf ein Bauteil öffnet seinen inneren Aufbau:

- **Flipflops, Register, Zähler, Addierer, Multiplexer …** → Aufbau aus Logikgattern,
- **Logikgatter** → Aufbau aus **CMOS-Transistoren** (der Reiter wechselt dafür automatisch zu **Elektronik & Halbleiter**),
- **eigene Bausteine** → die gespeicherte Schaltung.

Die Eingänge der Innenschaltung übernehmen die aktuellen Werte. Man kann sie verändern und die Innenschaltung so direkt testen – Änderungen werden beim Verlassen wieder verworfen. Eine Navigationszeile oben zeigt, wo man sich befindet; ← Zurück führt eine Ebene nach oben, der Knopf Erklärung blendet den Erklärungstext ein und aus.

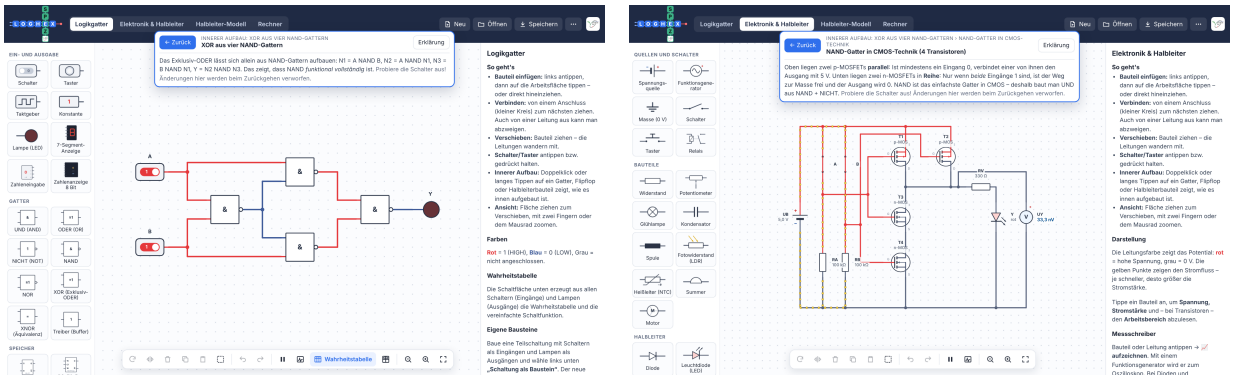


Abbildung 2.1.: Hineinzoomen: Das XOR-Gatter besteht aus vier NAND-Gattern (links), ein NAND-Gatter aus vier CMOS-Transistoren (rechts).

Tipp für den Unterricht

Ein schöner Weg durch die Ebenen: Volladdierer → XOR-Gatter → NAND-Gatter → Transistoren → (Reiter *Halbleiter-Modell*) pn-Übergang. So wird sichtbar, dass ein Rechenwerk letztlich nur aus geschickt verschalteten Halbleitern besteht.

2.5. Eigene Bausteine (modulares Arbeiten)

Jede Schaltung mit Schaltern (Eingänge) und Lampen (Ausgänge) kann zu einem **eigenen Baustein** werden:

1. Schaltung bauen und die Schalter und Lampen sinnvoll benennen (die Namen werden zu Anschlussnamen).
2. In der Bauteilleiste unter *Eigene Bausteine* → *Schaltung als Baustein* wählen und einen Namen vergeben.
3. Der neue Baustein steht nun in der Bauteilleiste und kann **beliebig oft** verwendet werden – auch **innerhalb anderer Bausteine** (z. B. Halbaddierer → Volladdierer → 4-Bit-Addierer).

4. Bearbeiten: Baustein auswählen → *Bearbeiten*; die Änderung gilt für alle Verwendungen.

Eigene Bausteine werden mit der Schaltung gespeichert und mitgeladen.

2.6. Wahrheitstabelle, KV-Diagramm und Minimierung

Der Knopf Wahrheitstabelle erzeugt aus allen Schaltern (Eingänge) und Lampen (Ausgänge) die vollständige Wahrheitstabelle. Für jeden Ausgang zeigt LOGHEX die **disjunktive Minimalform** und das **KV-Diagramm** mit farbig eingekreisten Blöcken (bis 4 Variablen). Die Tabelle lässt sich als CSV herunterladen, und mit *Schaltung daraus neu aufbauen* entsteht die minimierte Schaltung.

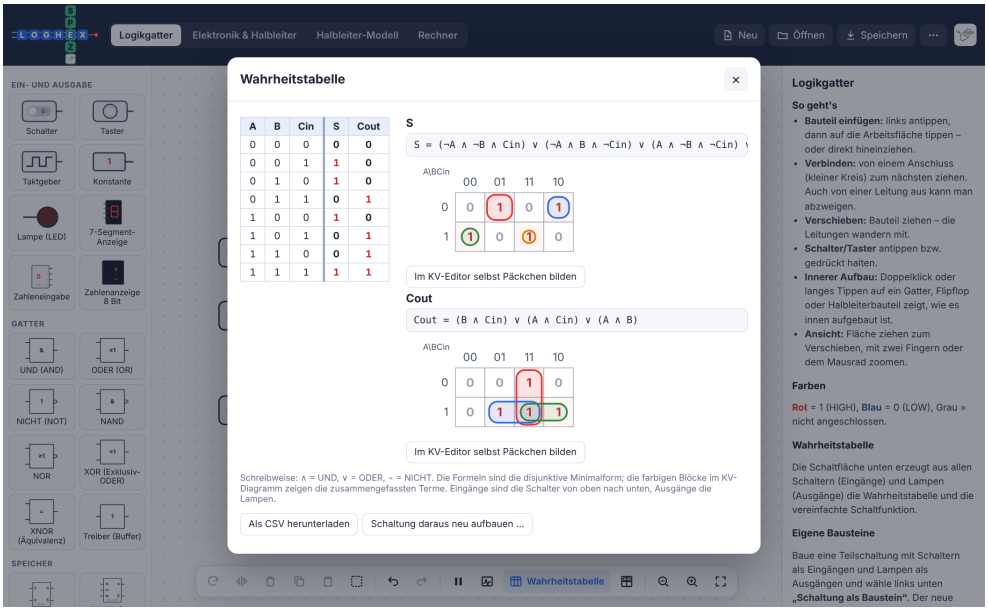


Abbildung 2.2.: Wahrheitstabelle und KV-Diagramme eines Volladdierers.

2.7. KV-Editor: Päckchen selbst bilden

Der Knopf KV-Diagramm in der Werkzeugleiste öffnet den KV-Editor. Anders als bei der Wahrheitstabelle fasst hier nicht LOGHEX zusammen: Die Schülerinnen und Schüler bilden die Päckchen selbst und schreiben auch die Terme und die Normalform selbst auf. LOGHEX zeigt keinen fertigen Term an.

- **Größe:** 2 × 2 (2 Variablen), 2 × 4 (3 Variablen) oder 4 × 4 (4 Variablen). Die Variablennamen und der Name des Ausgangs lassen sich ändern.
- ① **Werte eintragen:** Antippen wechselt ein Feld zwischen 0, 1 und x (beliebig, „don't care"); Ziehen überträgt den Wert auf weitere Felder. Daneben steht die Wahrheitstabelle, die mitläuft und ebenfalls angetippt werden kann. Die kleinen Zahlen in den Feldern sind die Zeilennummern der Wahrheitstabelle. Werte lassen sich auch zufällig erzeugen oder aus der aktuellen Schaltung übernehmen.

- ② **Päckchen bilden:** Über die Felder ziehen markiert ein Rechteck. Päckchen über den Rand oder die vier Ecken entstehen durch einzelnes Antippen der Felder. Schon während des Markierens steht darunter, ob die Auswahl ein gültiges Päckchen ist – und wenn nicht, warum (Anzahl keine Zweierpotenz, kein Rechteck aus Nachbarfeldern, enthält eine 0 …). Päckchen bilden übernimmt es.
- **Einsen oder Nullen:** Wahlweise werden Einsen zusammengefasst (DNF) oder Nullen (KNF).
- **Terme selbst aufschreiben:** Neben jedem Päckchen steht ein Eingabefeld für seinen UND-Term (DNF) bzw. seine ODER-Klausel (KNF), darunter ein Feld für die ganze Normalform `Y = …`. Für das iPad gibt es Knöpfe für ¬, ∧, ∨ und Klammern; alternativ gehen `NICHT/UND/ODER`, `! & |`, `+ *` oder `A'` für ¬A. Zusammengeschriebene Variablen wie `BD` werden als B ∧ D gelesen.
- Prüfen meldet
  - bei den Päckchen: nicht abgedeckte Einsen bzw. Nullen (rot umrandet), Päckchen, die sich noch **vergrößern** lassen, **überflüssige** Päckchen und ob es mit weniger Päckchen oder kürzeren Termen geht;
  - bei den Termen: ob jeder Term **genau sein Päckchen** beschreibt (✓ oder × neben dem Feld), ob er als UND-Term bzw. ODER-Klausel geschrieben ist, in welchen Zeilen der Wahrheitstabelle die Normalform einen falschen Wert liefert, ob sie wirklich eine DNF bzw. KNF ist und ob sie zu den eigenen Päckchen passt.

  Die Lösung selbst verrät LOGHEX nicht.
- Schaltung daraus aufbauen übergibt den selbst geschriebenen Term an die Schaltungssynthese (Kap. 2.8).

Aus dem Fenster *Wahrheitstabelle* führt der Knopf *Im KV-Editor selbst Päckchen bilden* direkt zum KV-Editor – mit den Werten der Schaltung, aber ohne Päckchen.

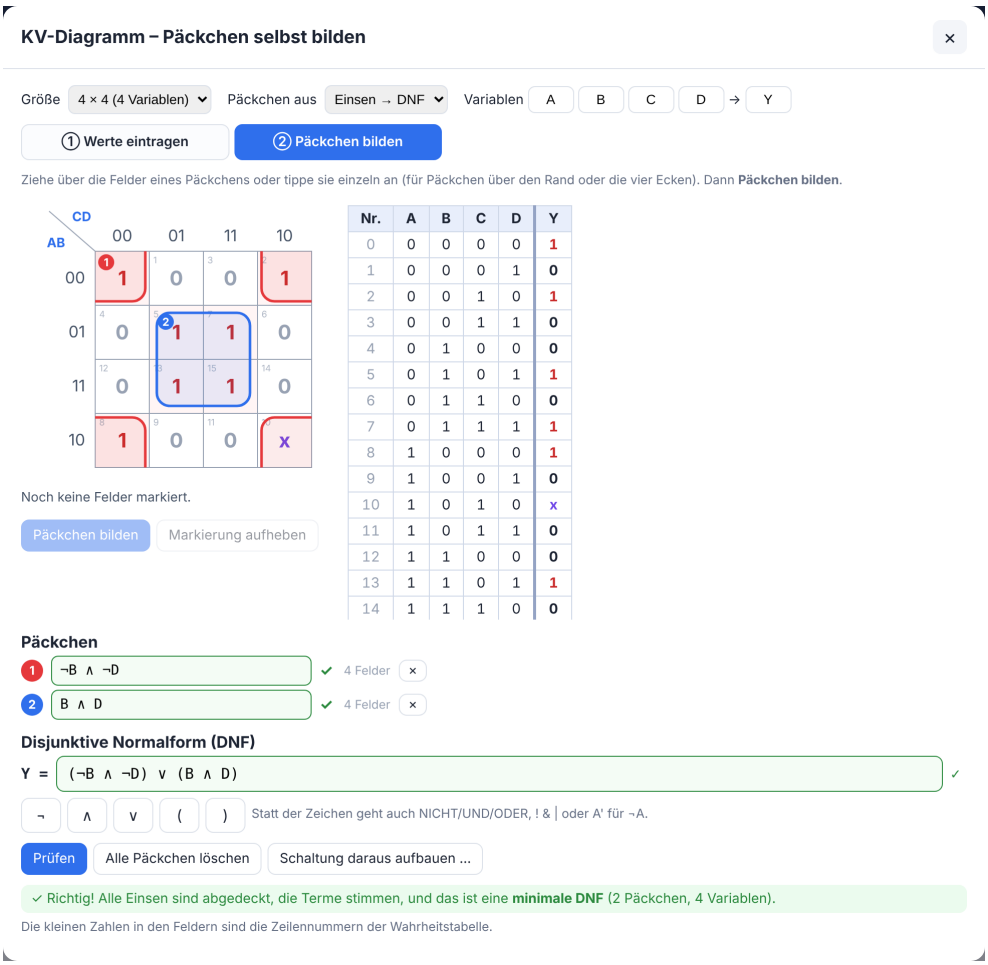


Abbildung 2.3.: KV-Editor mit zwei selbst gebildeten Päckchen (eines über die vier Ecken) und selbst geschriebener DNF.

Unterrichtsidee: Erst selbst, dann vergleichen

Die Schüler bauen eine Schaltung, öffnen über die Wahrheitstabelle den KV-Editor und bilden dort die Päckchen. Erst nach Prüfen vergleichen sie mit dem automatisch minimierten KV-Diagramm der Wahrheitstabelle. Mit *Zufällige Funktion* entstehen beliebig viele Übungsaufgaben.

2.8. Schaltung aus einem Term erzeugen

Unter ⋯ → *Schaltung aus Term erzeugen* gibt man eine oder mehrere Schaltfunktionen ein, z. B.

```
Y = A & B | !C
S = A xor B
```

Erlaubt sind `&` bzw. `*` (UND), `|` bzw. `+` (ODER), `!` bzw. nachgestelltes `'` (NICHT), `xor` bzw. `^` sowie Klammern und die Wörter *und, oder, nicht*. Die Schaltung kann wahlweise aus

UND/ODER/NICHT (mit XOR), **nur aus NAND** oder **nur aus NOR** aufgebaut werden – so lässt sich die Vollständigkeit dieser Gatter zeigen.

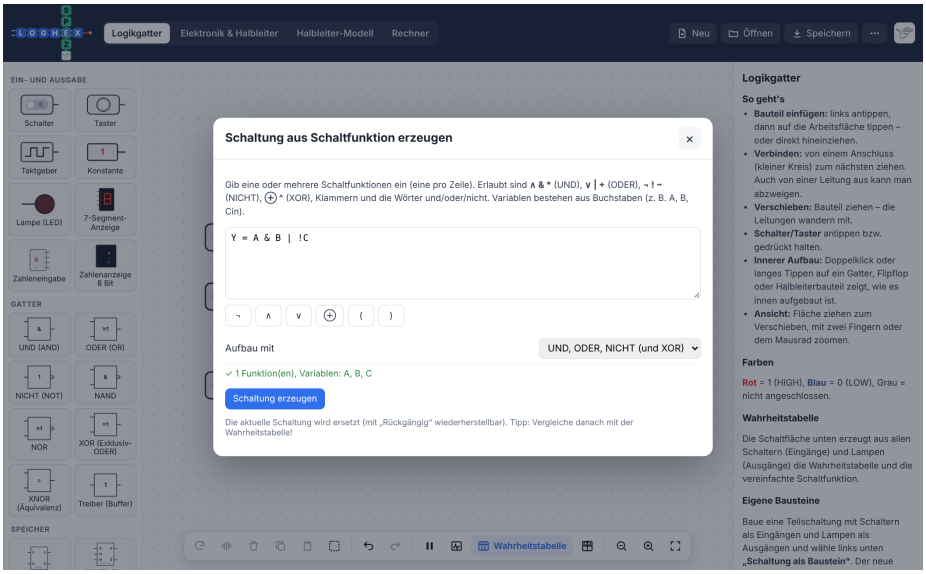


Abbildung 2.4.: Synthese einer Schaltung aus einem Term.

2.9. Impulsdiagramm (Messschreiber)

Bauteil oder Leitung antippen → aufzeichnen. Unten erscheint das Impulsdiagramm, das alle gewählten Signale über der Zeit zeigt. Das Zeitfenster ist einstellbar; die Daten können als CSV exportiert werden.

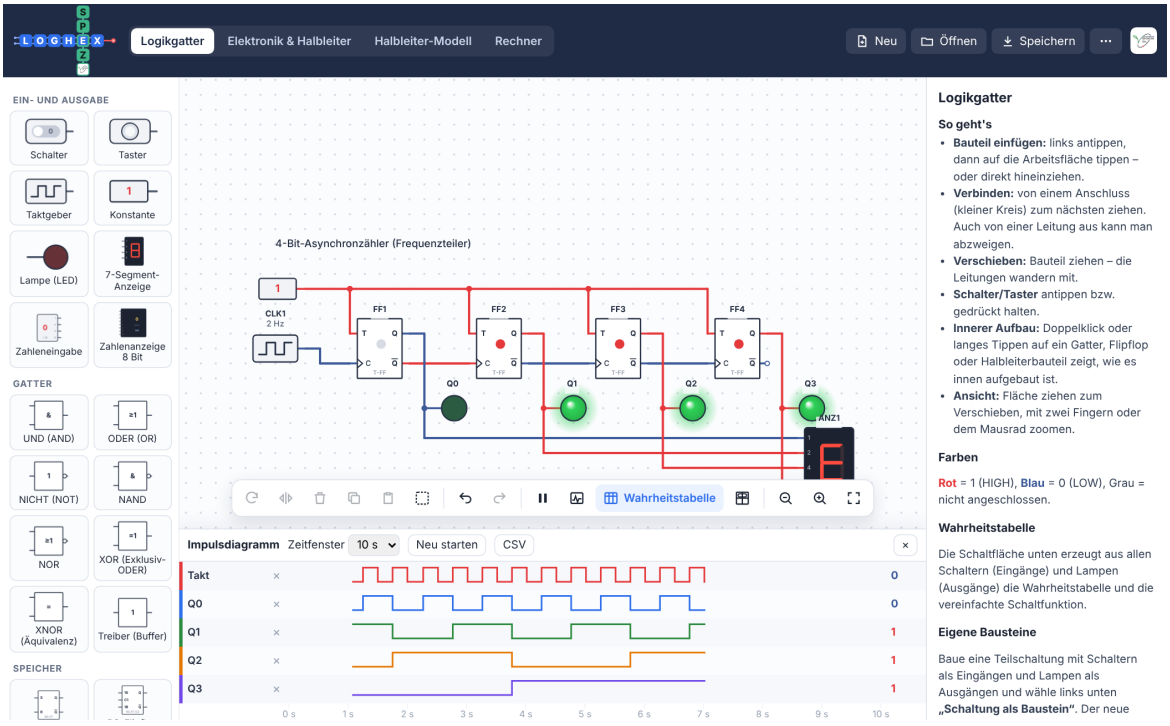


Abbildung 2.5.: 4-Bit-Asynchronzähler mit Impulsdiagramm: Jede Stufe halbiert die Frequenz.

2.10. Aufgabenmodus mit automatischer Prüfung

1. Die Lehrkraft baut eine **Musterlösung** (Schalter und Lampen benennen!).
2. ⋯ → *Aufgabe aus dieser Schaltung erstellen*: Titel, Aufgabentext, **erlaubte Bauteile**, eine **Höchstzahl an Bauteilen**, ob Schalter und Lampen vorgegeben werden und ob die Wahrheitstabellen-Synthese gesperrt wird.
3. Es entsteht eine **Aufgabendatei**. Die Lösung ist darin nur als Wahrheitstabelle enthalten, nicht als Schaltung.
4. Die Schüler öffnen die Datei, bauen ihre Schaltung und drücken Prüfen. LOGHEX vergleicht Soll- und Istwerte für alle Eingangskombinationen und markiert abweichende Zeilen.

2.11. Weitere Werkzeuge

- **Mehrfachauswahl:** Umschalt+Ziehen oder Knopf Auswahl; die Auswahl gemeinsam verschieben, kopieren, einfügen, löschen.
- **Leitungspunkte verschieben:** Knickpunkte einer Leitung lassen sich ziehen.
- **Zeitlupe** (Einstellungen): macht Gatterlaufzeiten sichtbar.

3. Elektronik & Halbleiter

3.1. Bauteile

Quellen, Schalter	Spannungsquelle, Funktionsgenerator (Sinus, Rechteck, Dreieck), Masse, Schalter, Taster, Relais
Bauteile	Widerstand, Potentiometer, Glühlampe, Kondensator, Spule, Fotowiderstand (LDR), Heißleiter (NTC), Summer, Motor
Halbleiter	Diode, Leuchtdiode, Z-Diode, NPN- und PNP-Transistor, n- und p-Kanal-MOSFET, Operationsverstärker, Timer 555
Messgeräte	Strommesser, Spannungsmesser

3.2. Simulation und Darstellung

Analoge Schaltungen werden wie in SPICE berechnet: Knotenpotentialanalyse mit dem Newton-Verfahren, Dioden nach der Shockley-Gleichung, Bipolartransistoren nach Ebers-Moll, MOSFETs mit dem quadratischen Modell, Kondensatoren und Spulen zeitabhängig. Die **Leitungsfarbe** zeigt das Potential (rot = hohe Spannung), die **gelben Punkte** den Stromfluss – je schneller, desto größer der Strom. Überlastete Bauteile brennen durch und können repariert werden; Kurzschlüsse werden gemeldet.

Tippt man ein Bauteil an, zeigt die Seitenleiste Spannung, Strom und Leistung bzw. den Arbeitspunkt.

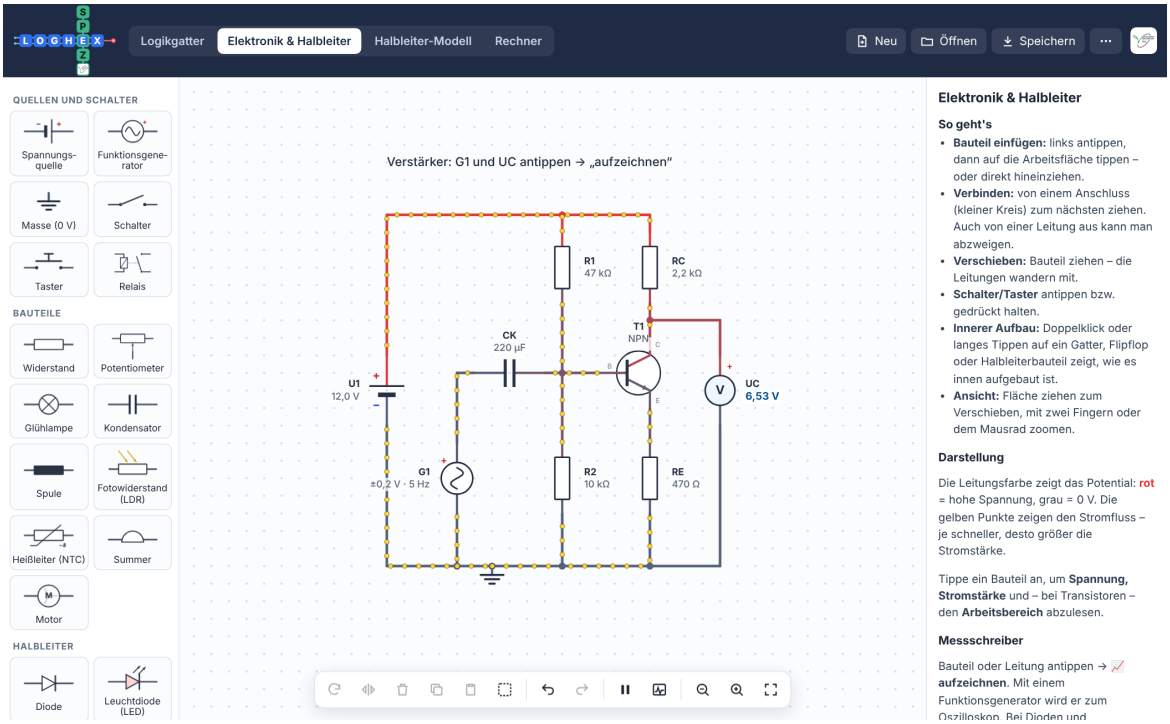


Abbildung 3.1.: Transistorverstärker in Emitterschaltung mit Funktionsgenerator.

3.3. Kennlinien und Oszilloskop

Für Dioden und Transistoren öffnet Kennlinie die Kennlinienfelder (beim Transistor: Ausgangs-, Eingangs- und Steuerkennlinie), berechnet mit demselben Modell wie die Simulation. Der **Messschreiber** wird mit einem Funktionsgenerator zum **Oszilloskop** für Spannungen und Ströme.

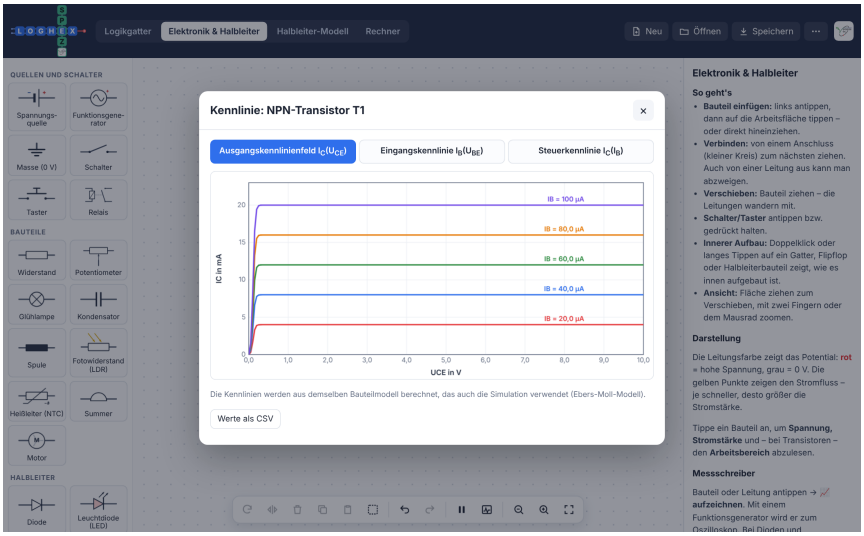


Abbildung 3.2.: Ausgangskennlinienfeld eines NPN-Transistors.

3.4. Innerer Aufbau

Auch hier zeigt ein Doppelklick bei geeigneten Bauteilen, wie sie aufgebaut sind bzw. funktionieren. Logikgatter aus dem Reiter **Logikgatter** erscheinen hier als CMOS-Schaltung mit echten Transistoren.

4. Halbleiter-Modell

Der Reiter **Halbleiter-Modell** veranschaulicht im **Teilchenmodell**, was in einem Halbleiter passiert:

- **Eigenleitung:** Durch Wärme entstehen Elektron-Loch-Paare, die wieder rekombinieren.
- **n-Dotierung** (Phosphor, Donatoren) und **p-Dotierung** (Bor, Akzeptoren).
- **pn-Übergang:** Diffusion, Rekombination und Entstehung der **Raumladungszone**. Mit dem Schieberegler für die Spannung lassen sich **Durchlass-** und **Sperrrichtung** untersuchen; die Kennlinie wird mitgezeichnet.
- Verschiedene Bauteile (Si-Diode, Ge-Diode, LED, Z-Diode) mit passender Kennlinie.

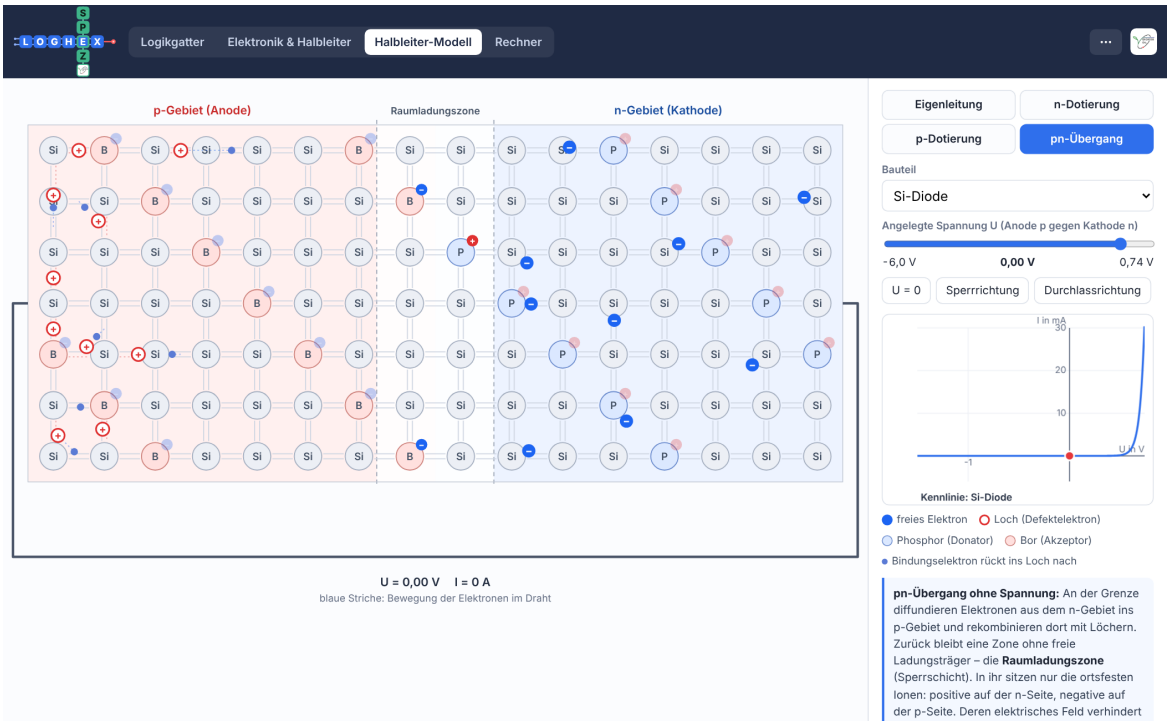


Abbildung 4.1.: pn-Übergang im Teilchenmodell mit Kennlinie.

4.1. Wie sich Löcher bewegen

Freie Elektronen bewegen sich im Modell frei durch den Kristall. Ein **Loch** dagegen ist kein eigenes Teilchen, sondern eine **Bindung, der ein Elektron fehlt**: Im Bild fehlt dort einer der beiden Striche (gestrichelt rot). Das Loch sitzt immer auf einer Bindung und wandert nur,

indem ein **Elektron aus einer Nachbarbindung nachrückt** – zu sehen als kleiner blauer Punkt, der in die Lücke springt, während das Loch in die Gegenrichtung weiterrückt. Bewegt werden also immer gebundene Elektronen, Schritt für Schritt. Ortsfest sind die Bor-Ionen (blaues –) und die Phosphor-Ionen (rotes +).

Mit angelegter Spannung rücken die Bindungselektronen bevorzugt in Richtung Pluspol nach, die Löcher wandern deshalb zum Minuspol. Im Bändermodell beschreibt man dieses Nachrücken vereinfacht als **bewegliche positive Ladung**; dass es sie gibt, zeigt z. B. der Hall-Effekt an p-dotiertem Silizium.

4.2. Warum die Raumladungszone in Durchlassrichtung schmaler wird

Die Breite der Raumladungszone hängt von der Spannung über ihr ab, ungefähr wie $w \sim \sqrt{U_\mathrm{D} - U}$ (U_D: Diffusionsspannung, U: äußere Spannung, positiv in Durchlassrichtung). Die äußere Spannung wirkt dem Feld der Ionen entgegen und schiebt Elektronen und Löcher in die Zone hinein: In Durchlassrichtung wird sie schmaler, in Sperrrichtung breiter. Entscheidend für den Strom ist dabei nicht, dass die Elektronen „mehr Kraft“ bekommen, sondern dass die **Energiebarriere sinkt**: Von den thermisch unterschiedlich schnellen Elektronen kommen dann exponentiell mehr hinüber – daher die exponentielle Kennlinie. Dass sich die Sperrschichtbreite wirklich ändert, nutzt die **Kapazitätsdiode**: Ihre Kapazität hängt von der angelegten Sperrspannung ab.

5. Rechner

5.1. Anfänger- und Profi-Ansicht

Unten links in der Leiste des Rechners schaltet Anfänger | Profi zwischen zwei Ansichten um. Die Einstellung bleibt auf dem Gerät gespeichert; das Programm im Speicher und alle Ergebnisse sind in beiden Ansichten gleich.

Die **Anfänger-Ansicht** zeigt nur das Wesentliche der Von-Neumann-Architektur:

- den **Bus als eine Leitung** – orange, wenn etwas übertragen wurde; Pfeile zeigen die Richtung,
- das **Rechenwerk als einen Block** mit Akku A, der gerade ausgeführten Rechnung (z. B. „5 + 3 = 8“) und den Flags,
- vom Steuerwerk nur **Befehlszähler** und **Befehlsregister**; keine Tore, keine einzelnen Busleitungen, kein MAR, DR, B oder SP,
- Schritt führt eine ganze Phase aus: ① **Befehl holen** bzw. ② **Befehl ausführen**. Danach steht im Steuerwerk **in Worten**, was passiert ist („Liest 3 aus Adresse $14“, „Das Rechenwerk rechnet 5 + 3 = 8“, „Sprung: …“) und was als Nächstes kommt; die benutzten Speicherzellen sind markiert,
- in der Seitenleiste die Reiter *Speicher*, *Assembler*, eine einfache *Befehlsliste* und eine kurze *Hilfe*. Pseudocode, Compiler, Befehlssatz-Editor und der innere Aufbau aus Logikgattern erscheinen nicht.

Die **Profi-Ansicht** ist der vollständige Rechner, wie er in den folgenden Abschnitten beschrieben wird.

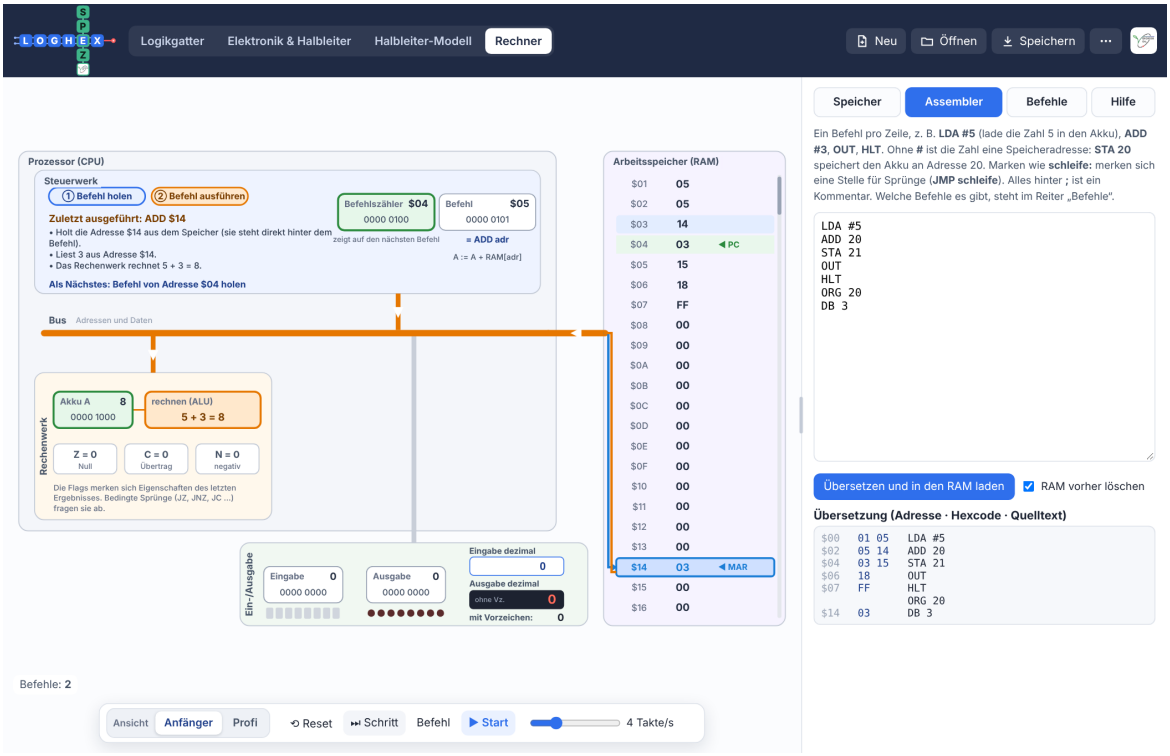


Abbildung 5.1.: Anfänger-Ansicht nach dem Befehl ADD: Was passiert ist, steht in Worten im Steuerwerk.

5.2. Aufbau

Der Reiter **Rechner** zeigt einen vollständigen 8-Bit-Rechner nach dem **Von-Neumann-Prinzip**: Programm und Daten liegen im selben Arbeitsspeicher (256 Byte). Zu sehen sind

- das **Steuerwerk** mit dem gerade laufenden **Mikroprogramm**,
- die Register **PC** (Befehlszähler), **SP** (Stapelzeiger), **IR** (Befehlsregister), **MAR** (Adressregister), **DR** (Datenregister),
- das **Rechenwerk** mit Akkumulator **A**, Hilfsregister **B**, **ALU** und den Flags **N, Z, C**. Unter der ALU ist zu sehen, woher die Flags kommen: N ist die Leitung von Bit 7 des Ergebnisses, Z liefert ein NOR-Gatter (≥1 mit Negation) an allen 8 Ergebnisleitungen, C kommt aus dem Übertragsausgang der ALU. Farbig sind diese Leitungen nur in Takten, in denen die Flags übernommen werden,
- der **interne Datenbus**, der **Adressbus**, der **Datenbus** und der **Steuerbus** (RD, WR),
- der **Arbeitsspeicher** (RAM) und die **Ein-/Ausgabe**.

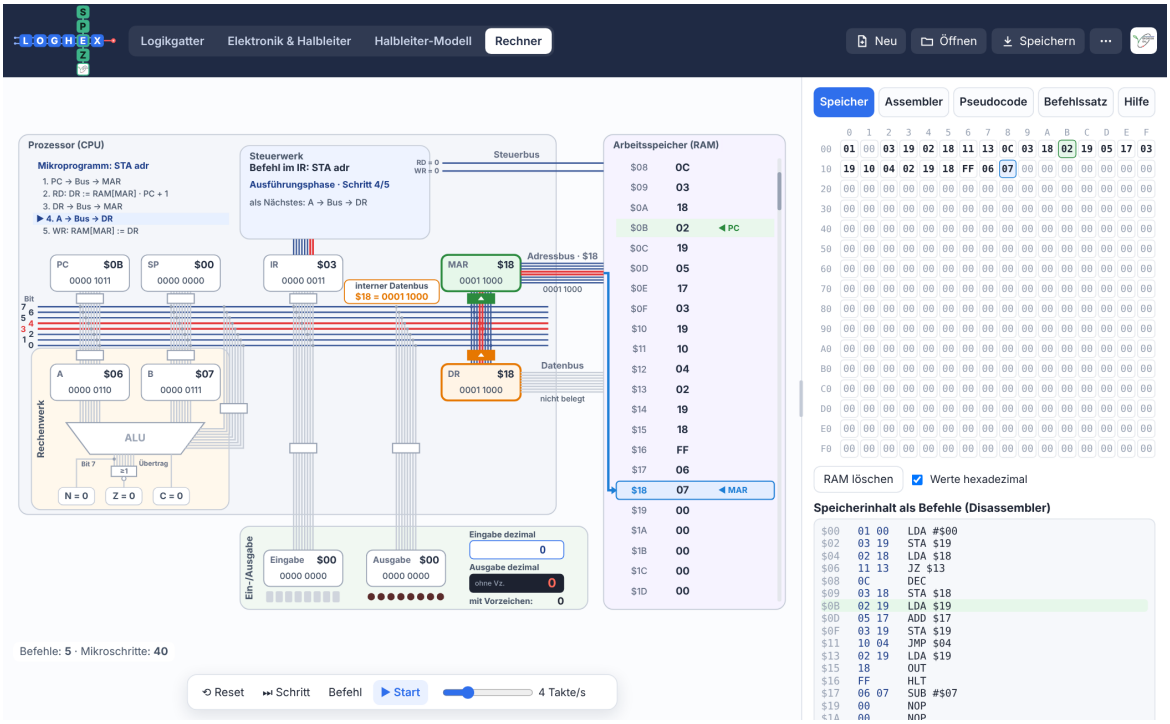


Abbildung 5.2.: Der Rechner bei der Ausführung eines Programms. Orange: Quelle auf dem Bus, grün: Register, die übernehmen.

5.3. Bitleitungen und Farben

Jeder Bus und jede Verbindung besteht aus **8 einzelnen Leitungen** – eine pro Bit (Bit 7 oben). **Rot** bedeutet 1, **blau** 0. Auch die beiden Eingänge der ALU sind als 8-Bit-Bündel dargestellt; die Ausgangsleitungen folgen der Form der ALU. So sieht man für jeden Befehl (z. B. `$A9`) genau, welche Leitungen aktiv sind. Die Register zeigen immer den Wert *nach* dem aktuellen Takt. Legt ein Register seinen Wert auf den Bus und wird im selben Takt weitergezählt (PC + 1, SP ± 1), steht über dem Register in Orange, welcher Wert auf dem Bus liegt – z. B. „auf dem Bus: $05, danach + 1“.

Am linken Ende des internen Busses steht für jede Leitung die **Bitnummer** (7 oben bis 0 unten), also die Zweierpotenz, für die sie steht. Jede Datenleitung eines Registers und der ALU ist mit einem Punkt an **ihre** Busleitung angeschlossen – Bit 7 an Bit 7 usw. So sieht man, wie ein Wert Bit für Bit vom Datenregister über den Bus in die Register und die ALU gelangt. Die **Tore** zwischen Registern und Bus zeigen den Steuerzustand: **orange** = das Register schreibt auf den Bus, **grün** = das Register übernimmt den Buswert. In der ALU erscheint das Symbol der aktuellen Operation (+, −, ∧, ∨, ⊕, ¬, ≪, ≫).

5.4. Steuerung

Schritt	genau **ein Takt** (ein Mikroschritt), dann wartet der Rechner. Tastatur: `Leertaste` oder `→`
Befehl	bis zum Ende des aktuellen Maschinenbefehls. Tastatur: `Eingabe`
Start	fortlaufend mit der eingestellten Geschwindigkeit (Takte pro Sekunde); Stopp oder `Leertaste` hält an
Reset	Register auf 0, PC = 0; der Speicher bleibt erhalten

Jeder Befehl läuft in zwei Phasen ab: In der **Holphase** (3 Takte) wird das Befehlsbyte über PC → MAR, RD und DR → IR geholt; in der **Ausführungsphase** läuft das Mikroprogramm des Befehls ab.

Die **Eingabe** besteht aus 8 Schaltern; daneben kann ein Wert **dezimal** eingegeben werden (0–255, negative Zahlen −128 bis −1 werden im Zweierkomplement gespeichert). Die **Ausgabe** zeigt 8 LEDs und den Dezimalwert, und zwar zweifach: **ohne Vorzeichen** (0–255) und **mit Vorzeichen** (−128 bis 127). Im **Zweierkomplement** steht das vorderste Bit für das Vorzeichen: Ist Bit 7 gesetzt, ist die Zahl negativ und um 256 kleiner. So ergibt $10 - 12$ das Bitmuster `1111 1110`, also 254 ohne und -2 mit Vorzeichen.

5.5. Arbeitsspeicher und Hexcode

Die **Seitenleiste** lässt sich an der Trennlinie mit dem Griff breiter oder schmaler ziehen (Doppelklick: Standardbreite). Im Reiter Speicher der Seitenleiste wird jede der 256 Zellen direkt als **Hexcode** bearbeitet. Darunter steht das Programm **disassembliert** als Befehle. Im Schaubild ist der RAM als **scrollbare Liste** dargestellt. Tippt man dort einen Wert an, lässt er sich direkt ändern (hexadezimal; `↑`/`↓` wechseln die Zelle, `Esc` bricht ab) – die Matrix in der Seitenleiste ändert sich mit; die Zelle, auf die der **Adressbus** gerade zugreift, ist hervorgehoben und mit dem Adressbus verbunden. „↻ mitlaufen“ lässt die Liste dem Zugriff folgen.

5.6. Assembler

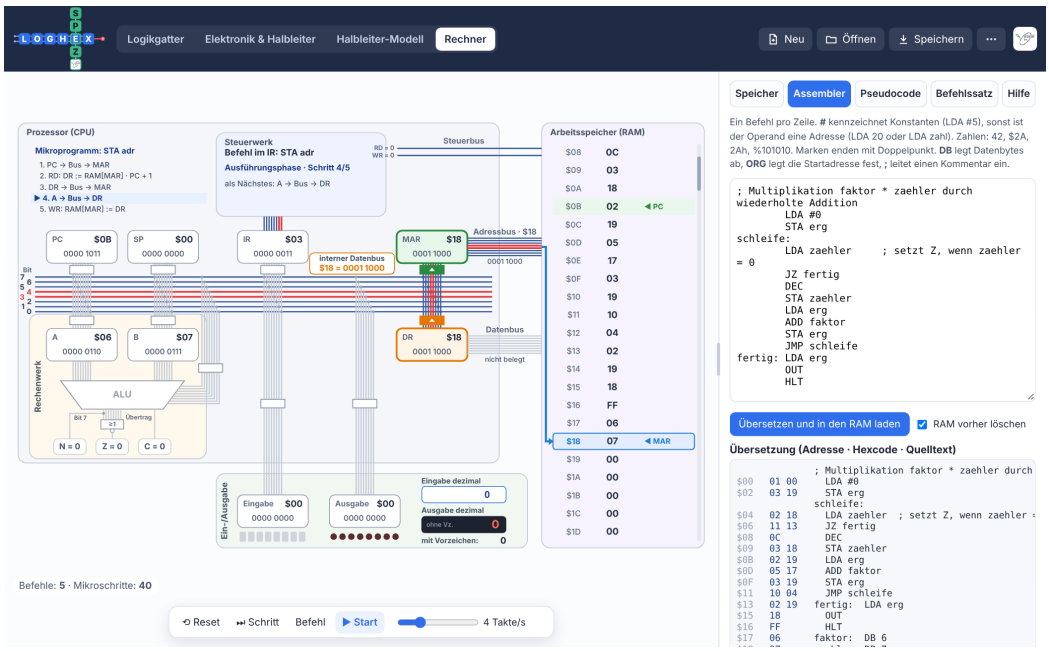


Abbildung 5.3.: Assemblerprogramm und übersetzter Hexcode.

Im Reiter Assembler schreibt man Programme in Textform; Übersetzen und in den RAM laden speichert sie als Hexcode im Speicher. Regeln:

- Ein Befehl pro Zeile; `;` leitet einen Kommentar ein.
- **Marken** enden mit Doppelpunkt (`loop:`) und können als Sprungziel oder Adresse benutzt werden; Rechnen mit Marken ist möglich (`lese+1`).
- `#` kennzeichnet eine Konstante (`LDA #10`); ohne `#` ist der Operand eine Adresse (`LDA zahl`).
- Zahlen: dezimal `42`, hexadezimal `$2A`, `2Ah` oder `0x2A`, binär `%00101010`, Zeichen `'A'`.
- `DB` legt Datenbytes ab (`liste: DB 12, 7, 30`), `ORG` setzt die Adresse.

```
; Multiplikation durch wiederholte Addition
        LDA #0
        STA erg
schleife:
        LDA zaehler     ; setzt Z, wenn zaehler = 0
        JZ fertig
        DEC
        STA zaehler
        LDA erg
        ADD faktor
        STA erg
        JMP schleife
fertig: LDA erg
        OUT
        HLT
```

```
faktor: DB 6
zaehler: DB 7
erg:    DB 0
```

5.7. Befehlssatz

Opcode	Befehl	Operand	Wirkung
$00	NOP	–	keine Operation
$01	LDA	#n	A := n (setzt Z und N)
$02	LDA	adr	A := RAM[adr] (setzt Z und N)
$03	STA	adr	RAM[adr] := A
$04 / $05	ADD	#n / adr	A := A + Operand
$06 / $07	SUB	#n / adr	A := A − Operand
$08	AND	#n	A := A ∧ n (bitweise)
$09	OR	#n	A := A ∨ n (bitweise)
$0A	XOR	#n	A := A ⊕ n (bitweise)
$0B	INC	–	A := A + 1
$0C	DEC	–	A := A − 1
$0D / $0E	CMP	#n / adr	Flags nach A − Operand setzen, A bleibt
$0F	SHL	–	A nach links schieben (·2), Bit 7 → C
$10	JMP	adr	Sprung nach adr
$11 / $12	JZ / JNZ	adr	Sprung, wenn Z = 1 / Z = 0
$13 / $14	JC / JNC	adr	Sprung, wenn C = 1 / C = 0
$15	JN	adr	Sprung, wenn N = 1
$18	OUT	–	Ausgabe := A
$19	IN	–	A := Eingabe
$1A / $1B	PUSH / POP	–	A auf den Stapel legen / vom Stapel holen
$1C	CALL	adr	Unterprogramm aufrufen (Rücksprungadresse auf den Stapel)
$1D	RET	–	Rücksprung aus dem Unterprogramm
$FF	HLT	–	Programm anhalten

Flags: Z = Ergebnis war 0; C = Übertrag bzw. „Borgen“ bei der Subtraktion (nach `CMP`: A < Operand); N = Bit 7 des Ergebnisses. **Stapel:** SP startet bei $00; PUSH und CALL verringern ihn zuerst, der Stapel wächst also von $FF abwärts.

5.8. Pseudocode-Compiler (Hochsprache)

Im Reiter Pseudocode der Seitenleiste schreibt man Programme in einer kleinen Hochsprache mit englischen Befehlen. Der eingebaute **Compiler** übersetzt sie in Assembler; der Assembler macht daraus Hexcode im RAM. So wird die ganze Kette sichtbar:

Pseudocode → Assembler → Hexcode → Mikroschritte → Gatter → Transistoren

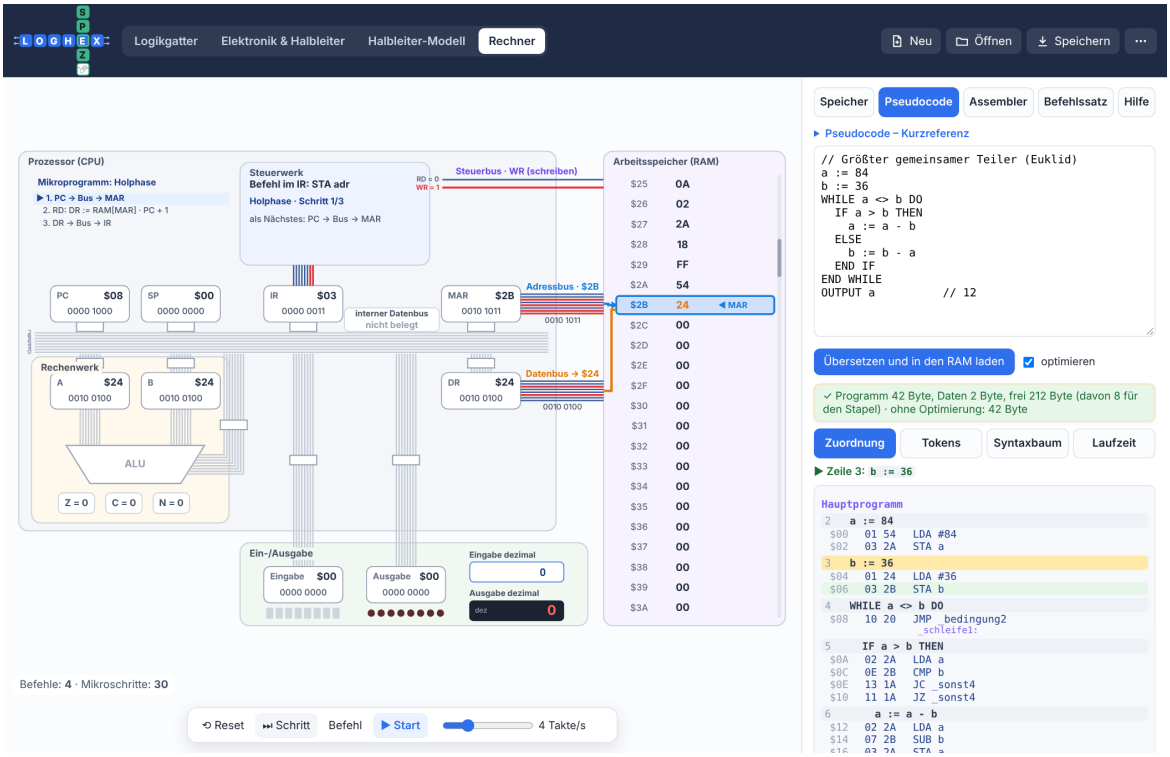


Abbildung 5.4.: Der ggT-Algorithmus im Schritt-Modus: Rechts ist die gerade ausgeführte Pseudocode-Zeile gelb und der zugehörige Maschinenbefehl grün markiert.

Sprachumfang

Groß- und Kleinschreibung der Befehle ist egal, Einrückungen dienen nur der Lesbarkeit. Pro Zeile steht eine Anweisung; `//` leitet einen Kommentar ein.

Schreibweise	Bedeutung
`x := 5` (auch `x ← 5`)	Zuweisung; Variablen entstehen bei der ersten Verwendung
`OUTPUT x` (auch `PRINT`)	Ausgabe (mehrere Werte mit Komma)
`INPUT x` oder `x := INPUT`	Eingabe von den Schaltern
`+ - * DIV MOD`	Grundrechenarten (ganzzahlig, 8 Bit)
`& | ^` (XOR)	bitweise; eine Seite muss konstant sein
`<< >>`	Schieben um eine konstante Anzahl Bits
`= <> < > <= >=`	Vergleiche (auch `!=`, ≠, ≤, ≥)
`AND OR NOT`, `TRUE FALSE`	logische Verknüpfungen
`IF … THEN … ELSE IF … ELSE … END IF`	Verzweigung
`WHILE … DO … END WHILE`	kopfgesteuerte Schleife
`REPEAT … UNTIL …`	fußgesteuerte Schleife
`FOR i := 1 TO 10 STEP 2 … END FOR`	Zählschleife, auch `DOWNTO` und `NEXT i`
`PROCEDURE name … END PROCEDURE`	Unterprogramm, Aufruf mit `CALL name`, vorzeitiges Ende mit `RETURN`
`ARRAY liste[5]`, `ARRAY z := [3, 1, 4]`	Feld anlegen, Zugriff mit `liste[i]`

Schreibweise	Bedeutung
`HALT`	Programm beenden

Was der Compiler daraus macht

```
summe := 0
i := 10
WHILE i <> 0 DO
  summe := summe + i
  i := i - 1
END WHILE
OUTPUT summe
```

```
        LDA #0
        STA summe
        LDA #10
        STA i
_while1:
        LDA i
        CMP #0
        JZ  _endewhile2
        LDA summe
        ADD i
        STA summe
        LDA i
        SUB #1
        STA i
        JMP _while1
_endewhile2:
        LDA summe
        OUT
        HLT
```

Dabei gelten einige Regeln, die die Grenzen der Maschine sichtbar machen:

- **Bedingungen** werden zu `CMP` und bedingten Sprüngen. Weil es nur die Flags Z und C gibt, braucht z. B. `a > b` zwei Sprünge.
- **Nur ein Akkumulator:** Bei verschachtelten Ausdrücken wie `(a+b) - (c+d)` legt der Compiler Zwischenergebnisse in Hilfsvariablen (`_t1`, `_t2` …) ab.
- `*`, `DIV` und `MOD` gibt es nicht als Maschinenbefehl. Der Compiler ruft dafür Unterprogramme der **Laufzeitbibliothek** auf (`_mul`: wiederholte Addition, `_div`: wiederholte Subtraktion). Ein einziges Zeichen in der Hochsprache kostet so viele Befehle.
- **Felder mit variablem Index:** Der Befehlssatz kennt keine indirekte Adressierung. Der Compiler berechnet die Adresse `liste + i` und schreibt sie zur Laufzeit in den Operanden des folgenden Befehls – **selbstverändernder Code**, möglich nur, weil Programm und Daten im selben Speicher liegen (Von-Neumann).
- Am Ende des Speichers bleiben 8 Byte für den **Stapel** frei (Rücksprungadressen von `CALL`). Ist das Programm zu groß, meldet der Compiler das.
- Hat jemand per Torsteuerung einen Befehl `SHR` gebaut (Kap. 5.9), benutzt der Compiler ihn automatisch für `>>`.

Die Ansichten

Zuordnung
Assembler in Programmreihenfolge, dazwischen die Pseudocode-Zeilen, jeweils mit Adresse und Hexcode. Beim Ausführen (Schritt, Befehl, Start) wird die aktuelle Zeile gelb und der aktuelle Maschinenbefehl grün markiert.

Tokens
Phase 1 des Compilers, die **lexikalische Analyse**: Der Text wird in Schlüsselwörter, Namen, Zahlen und Operatoren zerlegt.

Syntaxbaum
Phase 2, die **Syntaxanalyse**: Der Parser prüft die Grammatik und baut den Baum (Punkt- vor Strichrechnung). Die Codeerzeugung (Phase 3) durchläuft diesen Baum.

Laufzeit
Führt das Programm mit und ohne Optimierung im Hintergrund aus und vergleicht Programmgröße, Befehle und Takte.

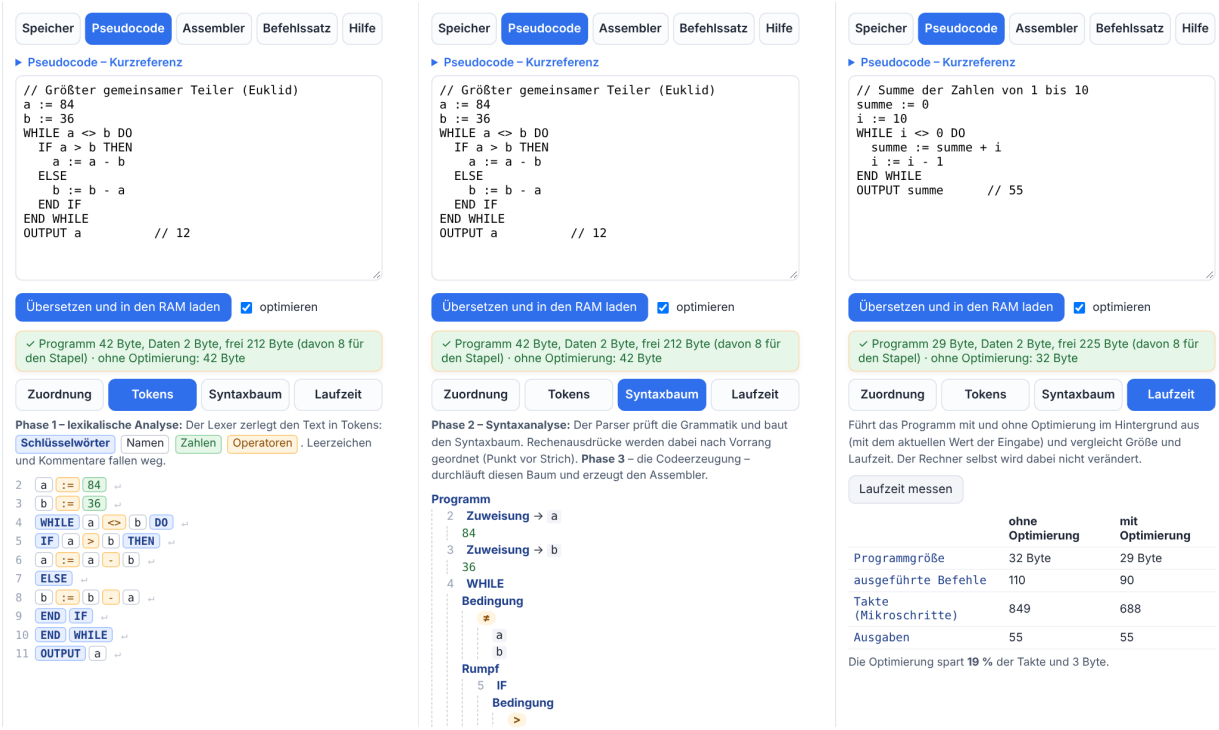


Abbildung 5.5.: Tokens, Syntaxbaum und Laufzeitvergleich.

Compiler-Labor: Schritt für Schritt zusehen

Der Knopf Compiler-Labor unter dem Pseudocode-Editor öffnet eine bildschirmfüllende Ansicht, in der man den drei Phasen des Compilers **Schritt für Schritt** zusehen kann. Oben steht der zu analysierende Text – ein Beispiel aus der Liste, das ganze Programm aus dem Editor oder nur die dort markierten Zeilen. Kurze Eingaben wie `x := (a + 3) * b - 1` eignen sich am besten.

Über jeder Phase steht ein **Infokasten** (abschaltbar mit Info): Er zeigt die ganze Übersetzungskette Quelltext → Tokens → Syntaxbaum → Assemblercode → Hexcode mit der aktuellen Phase und erklärt kurz Aufgabe, Eingabe und Ausgabe von Lexer, Parser bzw. Codeerzeugung.

Gesteuert wird wie beim Rechner: Schritt (→ oder Leertaste), ◀ (←), Abspielen mit einstellbarem Tempo. Unten erklärt ein Satz jeden einzelnen Schritt.

1 · Lexer
Ein **Lesekopf** wandert Zeichen für Zeichen durch den Text. Daneben zeigt ein **endlicher Automat** (Start, Name, Zahl, Operator, Kommentar), in welchem Zustand der Lexer gerade ist und welchen Übergang er nimmt. Ist ein Token fertig, wandert es in die Tokenliste. Man sieht, dass Leerzeichen und Kommentare überlesen werden und dass bei `:=` oder `<=` die **längste** passende Zeichenfolge gewinnt.

Schlüsselwort oder Name? Für den Automaten sehen `WHILE` und `summe` gleich aus – beides sind Buchstabenfolgen. Deshalb zeigt das Labor nach jedem fertigen Wort einen eigenen Schritt: Der Lexer schlägt das Wort (ohne Rücksicht auf Groß- und Kleinschreibung) in der **Tabelle der reservierten Wörter** nach. Wird es gefunden, leuchtet der Eintrag auf und das Token ist ein Schlüsselwort; sonst erscheint „nicht in der Tabelle → Name“. Daraus folgt auch, warum reservierte Wörter nicht als Variablennamen erlaubt sind.

2 · Parser
Ein **Lesezeiger** läuft über die Tokenliste. Die **Grammatik** in EBNF zeigt, welche Regel gerade arbeitet; der **Aufrufstapel** zeigt, welche Regel welche aufgerufen hat (rekursiver Abstieg). Auf der **Werkbank** entstehen Teilbäume von den Blättern her und werden zu immer größeren Bäumen zusammengesetzt. So wird sichtbar, *warum* bei `a + b * c` das Produkt tiefer im Baum hängt: Eine Summe besteht aus Produkten. Regeln, die nur weiterreichen, sind ausgeblendet; mit alle Grammatikregeln sieht man die vollständige Kette.

3 · Codeerzeugung
Der fertige Syntaxbaum wird durchlaufen. Der Knoten, der gerade übersetzt wird, ist gelb, wartende Knoten sind blau umrandet, fertige grün. Rechts erscheinen die Assemblerbefehle in dem Moment, in dem sie erzeugt werden – mit einer Erklärung, warum (z. B. Hilfsvariable, Unterprogramm `_mul`, Kurzschlussauswertung bei AND).

Der Stapel des Compilers: Die Codeerzeugung ist *rekursiv*: Um „+“ zu übersetzen, muss sie zuerst die Teilbäume übersetzen und ruft sich dafür selbst auf. In der Mitte zeigt der **Aufrufstapel**, welche Knoten gerade „offen“ sind und worauf sie warten (linker oder rechter Teil). Jeder Aufruf erscheint als eigener Schritt `PUSH`, jeder fertige Teilbaum als `POP` mit **Rücksprung** zum Elternknoten, der dann weitermacht – das Teilergebnis liegt im Akku. Das ist genau das Prinzip, mit dem der simulierte Rechner Unterprogramme mit `CALL` und `RET` über seinen Stapel aufruft.

Vorhersage-Modus: Ist er eingeschaltet, hält das Labor vor bestimmten Schritten an und fragt: *Welche Art Token ist das?* (Lexer), *Welcher Knoten wird als Nächstes gebaut?* (Parser) bzw. *Welcher Befehl wird als Nächstes erzeugt?* (Codeerzeugung). Erst nach der Antwort geht es weiter; oben wird mitgezählt, wie viele Vorhersagen richtig waren.

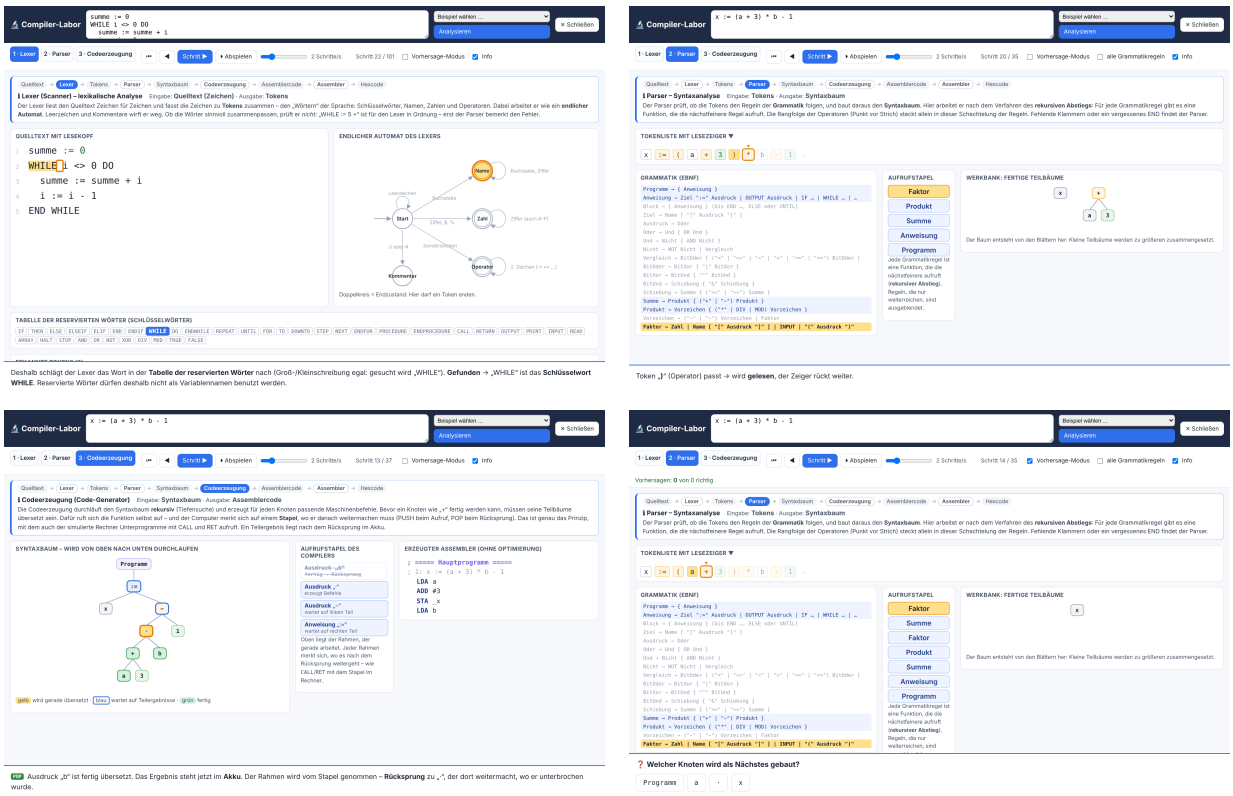


Abbildung 5.6.: Compiler-Labor: Lexer beim Nachschlagen in der Tabelle der reservierten Wörter, Parser mit Grammatik, Aufrufstapel und Werkbank, Codeerzeugung mit Stapel (POP/Rücksprung) sowie eine Frage im Vorhersage-Modus.

Unterrichtsidee: Mensch gegen Compiler

Am Beamer läuft der Parser im Vorhersage-Modus. Die Klasse stimmt vor jedem Schritt ab, welcher Knoten als Nächstes entsteht. Besonders lehrreich sind Ausdrücke mit Klammern und gemischten Rechenzeichen: Warum entsteht bei `(a + 3) * b - 1` zuerst das „+“, dann das „·“ und erst zuletzt das „−“?

Optimierung

Mit dem Häkchen optimieren wendet der Compiler typische Verbesserungen an: Konstanten werden schon beim Übersetzen ausgerechnet, `ADD #1` wird zu `INC`, ein `LDA x` direkt nach `STA x` entfällt, überflüssige `CMP #0` und Sprünge werden gestrichen, Multiplikation mit Zweierpotenzen wird zu `SHL`, `MOD` durch Zweierpotenzen zu `AND`, und `WHILE`-Schleifen werden „umgedreht“ (Bedingung am Ende), was pro Durchlauf einen Sprung spart.

Unterrichtsidee: Compiler spielen

Die Schüler übersetzen ein kurzes Pseudocode-Programm zuerst *von Hand* in Assembler und vergleichen dann mit dem Ergebnis des Compilers. Anschließend schalten sie die Optimierung ein und erklären jede Änderung.

5.9. Eigene Befehle per Torsteuerung

Im Reiter Befehlssatz ist jeder Befehl als Folge von **Mikroschritten** sichtbar. Jeder Mikroschritt ist ein Takt und legt fest,

- welche **Quelle** ihr Tor zum Bus öffnet (genau eine),
- welche **Register** den Buswert übernehmen (beliebig viele),
- ob gelesen (**RD**) oder geschrieben (**WR**) wird,
- welche **ALU-Operation** ausgeführt wird und ob die Flags gesetzt werden (die Operation „A (unverändert, nur Flags)“ setzt die Flags nach dem Inhalt von A, ohne etwas zu verändern),
- ob PC oder SP erhöht bzw. verringert werden,
- unter welcher **Bedingung** (Z, NZ, C, NC, N, nicht N) der Schritt wirkt.

Mit + Neuer Befehl entsteht ein neuer Opcode, den der Assembler sofort kennt. Prüfen deckt typische Fehler auf (Bus ohne Quelle, gleichzeitiges Lesen und Schreiben, Operand nicht übersprungen …). Mit dem **Schloss** wird ein fertiger Befehl **abgesichert**, sodass er nicht versehentlich verändert wird. Standard-Befehlssatz stellt den Ausgangszustand wieder her.

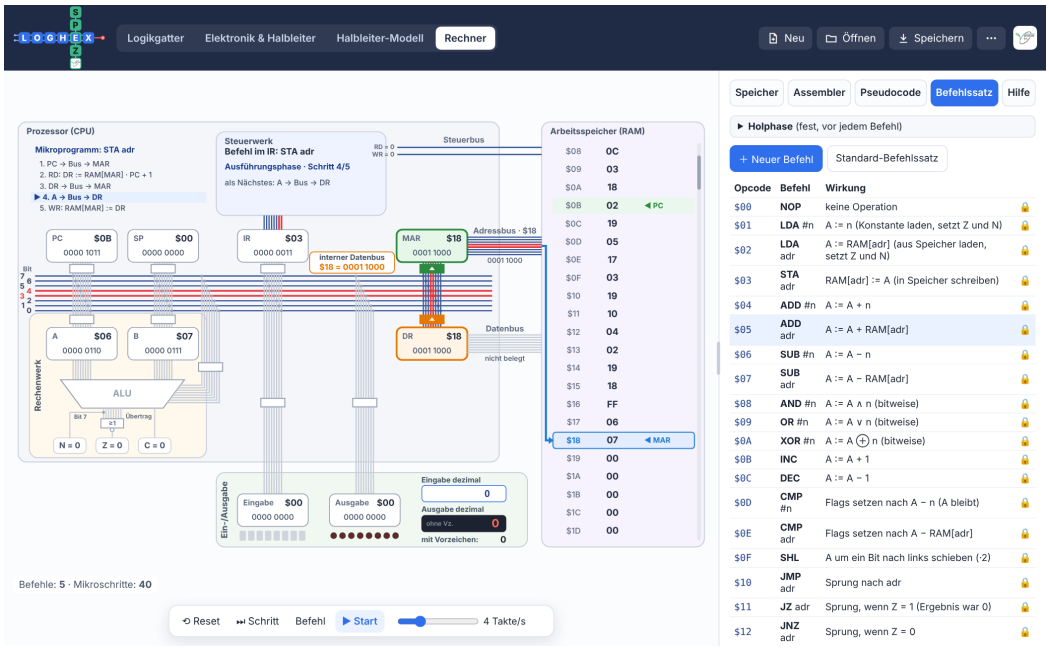


Abbildung 5.7.: Befehlssatz mit den Mikroschritten eines Befehls.

Unterrichtsidee: Ein Befehl fehlt

Der Standard-Befehlssatz enthält kein `SHR` (Rechtsschieben). Die Schüler bauen ihn selbst: ein einziger Mikroschritt „ALU: SHR, Quelle ALU, Ziel A, Flags setzen“. Das Beispielprogramm „Lauflicht hin und zurück“ enthält diesen selbst gebauten Befehl bereits.

5.10. MikroSim-Programme laden

Mit Öffnen lassen sich auch Dateien aus MikroSim laden:

- `.mpd` – das **Mikroprogramm**. Es legt fest, welcher Code welchen Befehl bedeutet. LOGHEX übernimmt daraus die Befehlsliste und schaltet auf einen passenden **MikroSim-Befehlssatz** um. Ohne eigene `.mpd`-Datei wird das verbreitete Mikroprogramm „CCAT-2014-03“ verwendet.
- `.ras` (RAM als Assemblerliste) und `.rad` (RAM als Hex-Tabelle) – das **Programm** mit seinen Daten. Es steht danach im RAM und im Reiter Assembler und kann sofort mit Start oder Schritt ausgeführt werden.

Der MikroSim-Rechner hat die Register `AX` und `BX`; `BX` ist zugleich Stapelzeiger und Indexregister. In LOGHEX übernimmt **A** die Rolle von `AX` und der **Stapelzeiger SP** die Rolle von `BX`; im Bild sind die Register entsprechend als „A (AX)“ und „SP (BX)“ beschriftet. Jeder MikroSim-Befehl wird aus LOGHEX-Mikroschritten nachgebaut und ist im Reiter Befehlssatz mit seiner MikroSim-Schreibweise zu sehen.

Im Assembler gilt dann die **MikroSim-Schreibweise**, z. B. `mov AX, [80]`, `ADD [81]`, `JPZ 10`. Zahlen ohne Präfix sind dort – wie in MikroSim – **hexadezimal**. Bedingte Sprünge prüfen wie in MikroSim den aktuellen Wert von `AX`.

Hinweis

Der Pseudocode-Compiler erzeugt LOGHEX-Assembler und funktioniert deshalb nur mit dem eigenen Befehlssatz. Mit Standard-Befehlssatz im Reiter Befehlssatz geht es zurück.

5.11. Erklärungen per Antippen

Ein Tipp auf ein Bauteil, einen Bus, ein Tor oder ein Flag öffnet ein Infofenster: **was die Abkürzung bedeutet** (z. B. MAR = *Memory Address Register*), **was das Bauteil tut** und – bei Registern und Flags – den **aktuellen Inhalt** (hexadezimal, dezimal und binär).

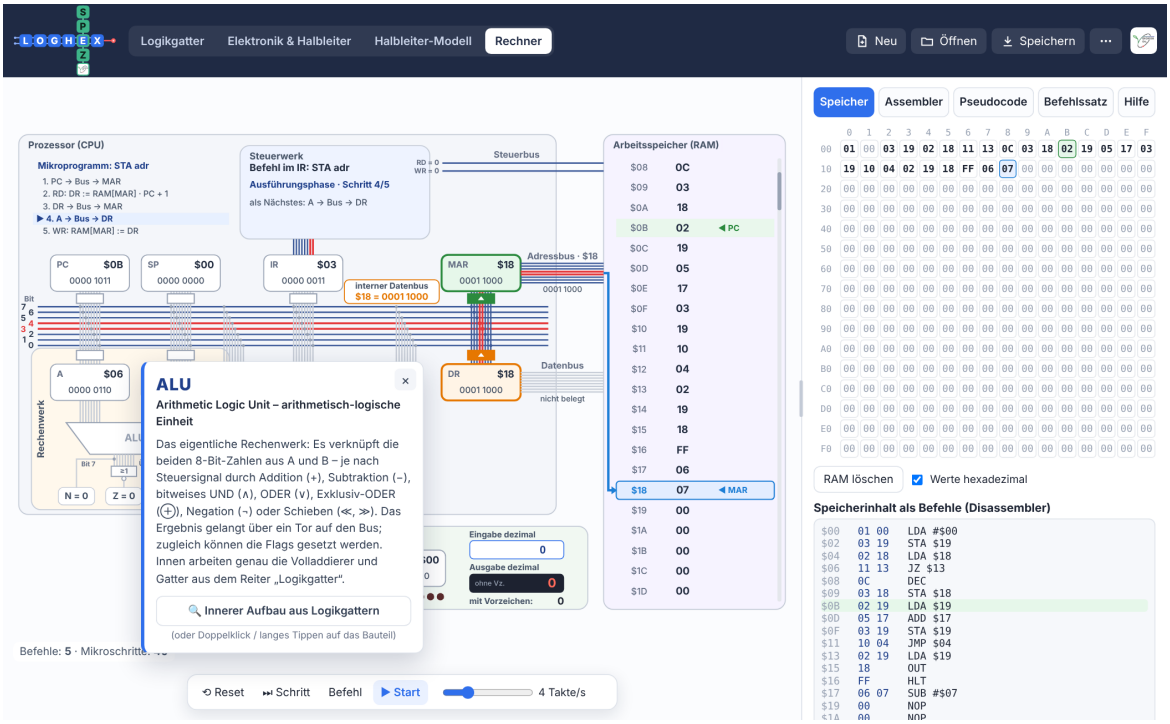


Abbildung 5.8.: Infofenster zur ALU mit Knopf für den inneren Aufbau.

5.12. Innerer Aufbau der Rechner-Bauteile

Ein **Doppelklick** (oder langes Tippen, oder der Knopf Innerer Aufbau aus Logikgattern im Infofenster) springt in den Reiter **Logikgatter** und zeigt das Bauteil als Gatterschaltung. Die Schaltung übernimmt die **aktuellen Werte aus dem Rechner** und kann mit Schaltern und Tastern **getestet** werden. ← Zurück führt wieder zum Rechner.

Element	innerer Aufbau
ALU, Rechenwerk	8-Bit-ALU aus acht **ALU-Scheiben** mit Übertragskette; Operation über S2 S1 S0 (+, −, ∧, ∨, ⊕, ¬, ≪, ≫); Ergebnis auf einer Dezimalanzeige. Jede Scheibe zeigt per Doppelklick Volladdierer, Gatter und Multiplexer.
A, B, IR, MAR, DR, OUT	8 D-Flipflops; der Takt erreicht sie über ein UND-Gatter nur bei „Laden"; dahinter das **Tor zum Bus** (8 UND-Gatter mit „Ausgeben").
PC	Register mit Halbaddierer-Kette (+1) und Multiplexern für Sprünge („Laden").
SP	Register mit Volladdierer-Kette für +1 und −1 (−1 = +255).
Z, C, N	Z über ODER- und NOR-Gatter, N = Bit 7, C mit XOR (Borgen bei SUB); in D-Flipflops gespeichert.
RAM, Adress-, Daten-, Steuerbus	RAM-Modell (4 Zellen × 4 Bit): Adressdecoder, Schreiben mit WR, Lesen über Multiplexer und RD.
Interner Bus, Tore, IN	Zwei Register über Tore an einem gemeinsamen Bus, Zielregister mit Anzeige (vereinfacht 4 Leitungen).

Element	innerer Aufbau
Steuerwerk, Mikroprogramm	Taktzähler und Decoder, die nacheinander die Steuersignale der Holphase einschalten (T0–T3).

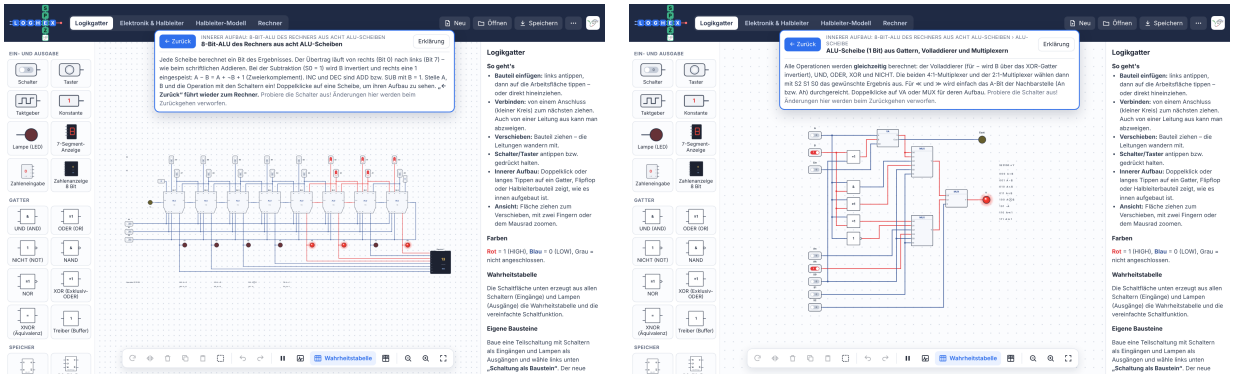


Abbildung 5.9.: Die 8-Bit-ALU des Rechners aus acht Scheiben (links) und eine einzelne ALU-Scheibe (rechts).

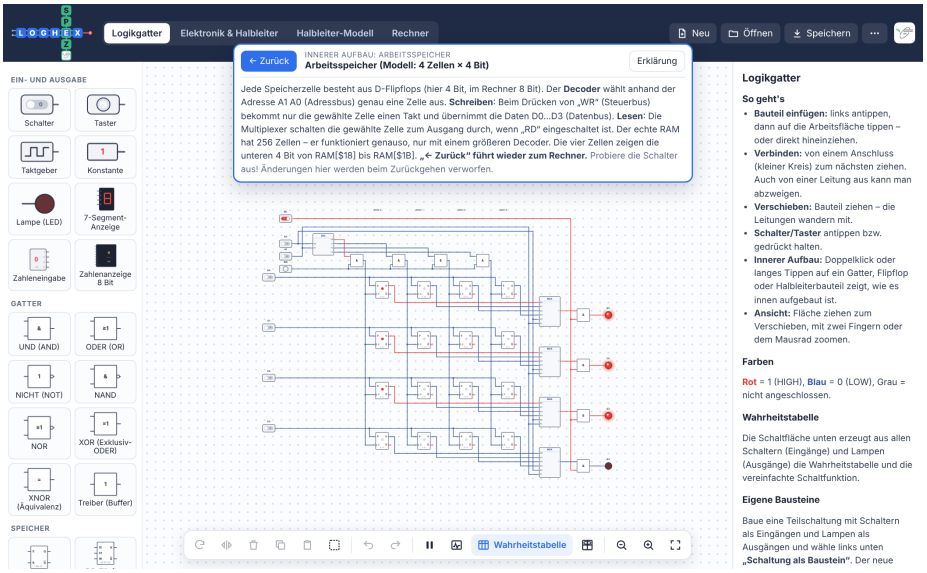


Abbildung 5.10.: Der Arbeitsspeicher als Gatterschaltung: Decoder, D-Flipflops und Multiplexer.

Vereinfachungen

Damit die Schaltungen übersichtlich bleiben, hat das RAM-Modell 4 Zellen statt 256 und der Bus 4 statt 8 Leitungen; das Steuerwerk zeigt nur die Holphase. Das Prinzip ist jeweils dasselbe wie im Rechner.

6. Material für die Lehrkraft

6.1. Beispieldateien laden und präsentieren

Im Programm selbst sind **keine fertigen Beispiele** enthalten – die Schüler sollen ihre Schaltungen selbst entwickeln. Die Beispiele liegen als eigene Dateien bei:

- `Beispielschaltungen.json` – alle Beispielschaltungen (Logikgatter und Elektronik) und alle Rechnerprogramme,
- `Rechner-Beispiele.json` – nur die Programme für den Rechner.

Mit Öffnen eine dieser Dateien wählen: Es erscheint eine Auswahl der Beispiele des aktuellen Reiters mit kurzer Beschreibung. Ein Klick lädt das Beispiel.

6.2. Beispielschaltungen

Logikgatter

- Halbaddierer, Volladdierer
- Alle Grundgatter
- RS-Flipflop aus NOR
- Volladdierer aus Bausteinen (modular)
- RS-Flipflops: ohne und mit Takt
- 4-Bit-Binärzähler
- D-Flipflop als 1-Bit-Speicher
- 4-Bit-ALU
- Akkumulator (Mini-Rechenwerk)
- Lauflicht (Johnson-Zähler)

Elektronik & Halbleiter

- LED mit Vorwiderstand
- Transistor als Schalter
- Dämmerungsschalter
- NICHT-Gatter aus Transistor
- Blinker (astabile Kippstufe)
- Z-Diode stabilisiert
- Kondensator laden und entladen
- Transistorverstärker (Emitterschaltung)
- Brückengleichrichter
- OPV: invertierender Verstärker
- Taktgeber mit Timer 555
- Selbstinduktion
- Relais schaltet einen Motor

6.3. Beispielprogramme für den Rechner

Programm	Inhalt und Lernziel
Addition zweier Zahlen	Laden, Addieren, Speichern – der Grundablauf.
Countdown mit Schleife	Zählt von 10 bis 0; bedingter Sprung mit dem Zero-Flag.
Multiplikation durch Addition	6 · 7 durch wiederholte Addition.
Unterprogramm mit CALL und RET	Rücksprungadresse auf dem Stapel (SP beobachten).
Eingabe verarbeiten	Endlosschleife: Ausgabe = Eingabe + 1.
Größere von zwei Zahlen	Vergleich mit CMP und dem Carry-Flag.
Lauflicht	Ein Licht wandert mit SHL über die LEDs.
Lauflicht hin und zurück	Nutzt den **selbst gebauten Befehl SHR** (eigener Befehlssatz).
Fibonacci-Zahlen	1, 1, 2, 3, 5, … bis zum 8-Bit-Überlauf (Carry).
Summe 1 + 2 + … + n	Zählschleife, Vergleich mit der Gauß-Formel (n = 10 → 55).
Quadratzahlen	n^2 als Summe ungerader Zahlen, ganz ohne Multiplikation.
Division mit Rest	47 : 5 = 9 Rest 2 durch wiederholte Subtraktion.
Gerade oder ungerade?	Bitmaske `AND #1` auf die Eingabe.
Einsen zählen	Anzahl der 1-Bits der Eingabe; SHL schiebt in das Carry-Flag.
Zahlenraten	Interaktiv: Die Ausgabe zeigt „zu klein“, „zu groß“ oder „richtig“.
Stapel: Reihenfolge umkehren	PUSH und POP – Last In, First Out.
Programm verändert sich selbst	Von-Neumann pur: Das Programm erhöht die Adresse in seinem eigenen ADD-Befehl.
Pseudocode (laden den Reiter „Pseudocode“ und übersetzen automatisch)	
Summe 1 bis 10	WHILE-Schleife; Vergleich mit und ohne Optimierung (Schleifenumkehr).
Countdown	FOR … DOWNTO.
Einmaleins der 7	* wird zum Unterprogramm `_mul`.
Division mit Rest	DIV und MOD über `_div`.
ggT nach Euklid	WHILE mit IF … ELSE.
Primzahlen	Verschachtelte Schleifen, MOD und AND.
Fibonacci	REPEAT … UNTIL, Ende beim Überlauf.
Bubblesort	ARRAY mit variablem Index – selbstverändernder Code.
Lauflicht mit Prozedur	PROCEDURE/CALL werden zu CALL/RET mit Stapel.
Zahlenraten	INPUT und IF … ELSE IF … ELSE in einer Endlosschleife.

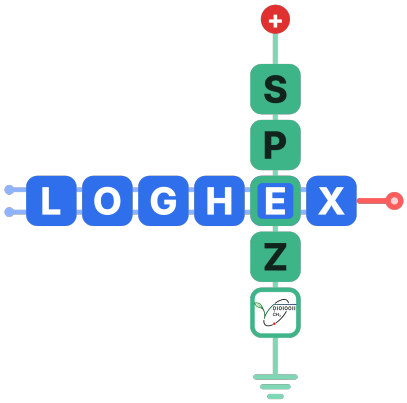
Unterrichtsidee: Von-Neumann-Flaschenhals

Lassen Sie das Programm „Multiplikation durch Addition“ im Schritt-Modus laufen und die Schüler die Takte zählen, die nur für das Holen von Befehlen und Operanden über den Bus gebraucht werden. Der gemeinsame Bus für Befehle und Daten wird so als Engpass erfahrbar.

7. Über LOGHEX

LOGHEX wurde von **Johannes Süpke** mit Hilfe von **Claude** (KI-Assistent von Anthropic) für den Unterrichtseinsatz im Fach Informatik erstellt.

MINT-Spezialklassen am Albert-Schweitzer-Gymnasium Erfurt



www.asgspez.de

Lizenz

© 2026 Johannes Süpke, MINT-Spezialklassen am Albert-Schweitzer-Gymnasium Erfurt.

- **Programm** (`LOGHEX.html`): Open-Source-Lizenz für die Europäische Union (EUPL) v. 1.2. Das Programm darf genutzt, vervielfältigt, verändert und weitergegeben werden. Veränderte Fassungen müssen wieder unter der EUPL weitergegeben werden. Die HTML-Datei ist zugleich der Quelltext.
- **Handbuch und Beispieldateien**: Creative Commons Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0). Bei der Weitergabe ist der Name des Urhebers zu nennen; Bearbeitungen sind unter derselben Lizenz weiterzugeben.
- **Ausgenommen** von beiden Lizenzen sind das Logo der MINT-Spezialklassen am Albert-Schweitzer-Gymnasium Erfurt und das LOGHEX-Logo, das dieses enthält. Die unveränderte Weitergabe von Programm und Handbuch mit Logo ist erlaubt; jede andere Verwendung des Logos bedarf der Erlaubnis.

Technische Hinweise

- Eine einzige HTML-Datei ohne externe Abhängigkeiten; läuft in allen aktuellen Browsern (Chrome, Edge, Firefox, Safari).
- Speicherung im Browser (*localStorage*); keine Daten verlassen das Gerät.
- Dateiformat: JSON. Schaltungen, eigene Bausteine, Aufgaben und Rechnerprogramme (inklusive eigener Befehlssätze) werden darin gespeichert.

A. Qualitätssicherung

A.1. Automatisierte Tests

LOGHEX wird mit einer Sammlung automatischer Tests geprüft, die in einem Browser ohne Bildschirm (Chromium) ablaufen. Alle Testgruppen werden nach jeder Änderung gemeinsam ausgeführt.

Testgruppe	Inhalt
Fachlogik	Alle Grundgatter mit 2–4 Eingängen in allen Kombinationen; Addierer, Multiplexer und Decoder erschöpfend (600 Fälle); Flipflops; CMOS-Innenschaltungen (Ausgangsspannung 0 V/5 V); innerer Aufbau aller Bausteine gegen das Verhalten des Bausteins (874 Fälle).
KV-Editor	Päckchen-Regeln (Ränder und Ecken als Nachbarn, falsche Größen, Formen und Inhalte); kleinstmögliche Lösung bei 240 Zufallsfunktionen gegen eine unabhängige Rechnung; Hinweise beim Prüfen; Bewertung selbst geschriebener Terme (falsche Terme, andere Schreibweisen, keine Normalform, unbekannte Variablen, DNF und KNF); Bedienung mit Maus und per Touch; manipulierte gespeicherte Zustände.
Werkzeuge	Synthese aus 45 zufälligen und festen Termen (normal, nur NAND, nur NOR); Minimierung von 60 Zufallsfunktionen; Aufgabenmodus (richtige und falsche Lösung).
Analogsimulation	Spannungsteiler (6,00 V) und LED-Strom (≈ 15 mA) gegen die Rechnung von Hand; alle Beispielschaltungen konvergieren.
MikroSim	Mikroprogramm einlesen, Euklid (ggT), Zweierpotenz und A4 mit Sollwerten; ein Programm mit allen Befehlen und 60 Zufallsprogramme gegen einen unabhängigen MikroSim-Interpreter.
Ansichten	Anfänger/Profi umschalten; Reiter, Schritt = Phase, Beschreibung in Worten, gleiche Ergebnisse in beiden Ansichten; Zeichnen ohne Fehler.
Rechner	17 Assembler-Programme mit Sollwerten; Standard-Befehlssatz fehlerfrei; alle Infofenster und alle 22 inneren Ansichten; Verdrahtung der inneren Ansichten (keine offenen Eingänge, keine losen Leitungen, keine Kurzschlüsse).

Testgruppe	Inhalt
Compiler	Alle Pseudocode-Beispiele und Grenzfälle; Hunderte zufällig erzeugte Programme, mit und ohne Optimierung, verglichen mit einer direkten Auswertung des Pseudocodes.
Dateien	Speichern/Laden und Teilen-Link verlustfrei; alle 78 Einträge der Beispieldateien; Zustand nach dem Neuladen.
Touch (iPad-Emulation)	Platzieren per Tippen, langes Tippen für den inneren Aufbau, Wischen, Antippen und Doppeltippen im Rechner, Knöpfe.
Sicherheit	Präparierte Dateien, Links und gespeicherte Zustände mit eingeschleustem HTML/JavaScript in *jedem* Textfeld (Namen, Beschriftungen, Bausteine, Aufgaben, Assembler, Pseudocode, Befehlssatz); Prototype Pollution.
Robustheit	500 zufällig beschädigte Dateien (fehlende Felder, falsche Datentypen, riesige Zahlen, doppelte IDs).
Überlastung	100 000 Bauteile, Texte mit 1 Mio. Zeichen, „Zip-Bombe“ im Teilen-Link (200 MB entpackt), sich selbst enthaltende Bausteine, tief verschachtelte Programme, Endlosschleifen.

A.2. Sicherheitsmaßnahmen

LOGHEX ist eine einzelne Datei ohne Server, ohne Anmeldung und ohne Datenübertragung; alle Daten bleiben im Browser des Geräts. Damit präparierte Dateien oder Links keinen Schaden anrichten können, gilt:

- **Eingabeprüfung:** Alles, was von außen kommt (Dateien, Links, gespeicherter Zustand), wird beim Laden geprüft: unbekannte Felder werden verworfen, Zahlen erzwungen, Texte gekürzt, Befehlssätze auf gültige Werte geprüft.
- **Maskierung:** Alle Texte werden vor der Anzeige maskiert, sodass HTML und JavaScript nur als Text erscheinen.
- **Content-Security-Policy:** Die Offline-Datei erlaubt nur ihr eigenes, per Prüfsumme festgelegtes Programm. Selbst wenn an einer Stelle das Maskieren fehlen sollte, würde ein eingeschleustes Skript vom Browser blockiert.
- **Obergrenzen:** höchstens 5 MB pro Datei, 3000 Bauteile, 12 000 Leitungen; entpackte Links höchstens 5 MB.

Grenzen

Im **Aufgabenmodus** steht die Lösung als Wahrheitstabelle in der Aufgabendatei. Wer die Datei mit einem Texteditor öffnet, kann sie lesen – für Übungen ist das unerheblich, für bewertete Leistungen ist der Aufgabenmodus nicht geeignet. Außerdem laufen die automatischen Tests in Chromium (Chrome, Edge); Safari auf dem iPad ist durch die Touch-Emulation nur angenähert. Deshalb gibt es die folgende Prüfliste.

A.3. Prüfliste für das iPad

Einmal vor dem ersten Einsatz auf einem Schul-iPad durchgehen (Safari, aktuelles iPadOS):

- ☐ LOGHEX über den Link öffnen und über *Teilen → Zum Home-Bildschirm* als App ablegen.
- ☐ Bauteil in der Leiste antippen und auf die Fläche tippen; ein zweites Bauteil hineinziehen.
- ☐ Zwei Anschlüsse durch Ziehen verbinden; Schalter antippen – die Lampe wechselt.
- ☐ Mit zwei Fingern zoomen und die Fläche verschieben.
- ☐ Lange auf ein Gatter tippen → innerer Aufbau; mit „← Zurück“ wieder heraus.
- ☐ Mehrere Bauteile mit „Auswahl“ markieren, kopieren und einfügen.
- ☐ Reiter „Elektronik & Halbleiter“: Beispiel „LED mit Vorwiderstand“ laden – Stromfluss-Animation läuft flüssig.
- ☐ Reiter „Halbleiter-Modell“: Animation läuft flüssig.
- ☐ Reiter „Rechner“: ALU antippen (Info), doppelt tippen (Gatteraufbau), „Schritt“ und „Start“; Dezimalzahl mit der Bildschirmtastatur eingeben.
- ☐ Reiter „Pseudocode“: Programm mit der Bildschirmtastatur ändern, übersetzen, Compiler-Labor öffnen und bedienen.
- ☐ „Speichern“ legt eine Datei in der App „Dateien“ ab; „Öffnen“ lädt sie wieder (auch per AirDrop erhaltene Dateien).
- ☐ „Link zum Teilen“ erzeugen und auf einem anderen Gerät öffnen.
- ☐ iPad ins Hochformat und zurück drehen – die Oberfläche passt sich an.
- ☐ Safari schließen und LOGHEX erneut öffnen – die letzte Arbeit ist noch da.
- ☐ Per AirPlay/Beamer im Beamer-Modus präsentieren – Schrift gut lesbar; Summer ist hörbar.